

INFORMATION SECURITY MANAGEMENT

Practical Take-Home Assessment

Identity Management using Keycloak

Centralised IAM for UniversityRealm: realm, users, RBAC, OIDC client and authentication demonstration

Student name	[Your full name]
Registration number	[Your registration number]
Programme	[Programme / Course]
Course unit	Information Security Management
Lecturer	[Lecturer name]
Practical tool	Keycloak 26.7.4 (OpenID Connect)
Total marks	40
Date of submission	[Submission date]

Table of Contents

1. Introduction	3
1.1 Objectives	3
2. Keycloak Environment and Setup (Task 1)	4
2.1 Environment used	4
2.2 Installation steps	4
3. Realm Configuration (Task 2)	8
3.1 Creating UniversityRealm	8
3.2 Realm security settings	9
4. User Configuration (Task 2)	11
4.1 Enabling and disabling a user account	13
5. Role Configuration (Task 3)	17
5.1 Creating the roles	17
5.2 Assigning roles to users	19
5.3 How least privilege is applied	22
5.4 Demonstrating different authorization privileges	23
6. Client Configuration (Task 4)	28
6.1 Authentication protocol: OpenID Connect	28
6.2 Redirect / allowed URLs	28
7. Authentication Demonstration (Task 4)	32
7.3 Authentication vs authorization in this implementation	37
8. IAM Architecture Diagram (Task 5A)	39
9. Security Analysis (Task 5B)	40
Risk 1: Weak, reused or stolen passwords (credential compromise)	40
Risk 2: Excessive privileges and orphaned accounts (privilege creep)	40
10. Recommendations (Task 5C)	41
Recommendation 1: Enforce Multi-Factor Authentication (MFA), starting with privileged roles	41
Recommendation 2: Harden the production deployment and connect Keycloak to the authoritative directory	41
11. Conclusion	42
12. References	43
Appendix A: Reproducing the Configuration	44
Appendix B: Marking-scheme cross-reference	44

1. Introduction

The institution is moving to a single, centralised Identity and Access Management (IAM) service built on **Keycloak**, an open-source Identity Provider that supports OpenID Connect (OIDC), OAuth 2.0 and SAML 2.0. The institution has three categories of users: **Administrators, Lecturers and Students**. Until now each application kept its own user accounts, so the same person had several passwords, there was no single place to revoke access, and permissions were inconsistent.

Acting as the **IAM Administrator**, I installed Keycloak and did the following:

- created the dedicated realm **UniversityRealm**;
- created the users **admin01**, **lecturer01** and **student01**;
- configured role-based access control with the roles **system-admin**, **lecturer** and **student**;
- registered an institutional application, **student-portal**, as an OIDC client;
- showed that each user authenticates through Keycloak and receives different authorization in the application.

The report follows the required structure. Every configuration step has screenshot evidence taken from the running system. The full configuration can also be reproduced from the accompanying scripts (Appendix A).

1.1 Objectives

Task	Objective	Section
1	Install and start Keycloak, create an administrative account, document the environment	2
2	Create UniversityRealm and three users with attributes; enable/disable accounts	3, 4
3	Create roles, assign them, apply least privilege, prove different privileges	5
4	Create an OIDC client, configure redirect URLs, demonstrate authentication	6, 7
5	Architecture diagram, security analysis and recommendations	8, 9, 10

2. Keycloak Environment and Setup (Task 1)

2.1 Environment used

Item	Value
Operating system	Ubuntu 24.04.4 LTS (Noble Numbat), x86_64, Linux kernel 6.18
Hardware (VM)	4 vCPU, 15 GiB RAM
Java runtime	OpenJDK 21.0.10
Keycloak version	26.7.4 (Quarkus distribution, keycloak-26.7.4.tar.gz)
Start mode	<code>kc.sh start-dev</code> , development profile, embedded H2 database, HTTP port 8080
Admin Console URL	<code>http://localhost:8080/admin</code>
Test application	Student Portal: Node.js 22, Express 4, openid-client 5, port 3000

2.2 Installation steps

1. Install the Java 21 runtime: `sudo apt install openjdk-21-jre-headless`.
2. Download the Keycloak 26.7.4 distribution and extract it: `tar xzf keycloak-26.7.4.tar.gz && cd keycloak-26.7.4`.
3. Start Keycloak with a temporary bootstrap administrator (Keycloak 26 creates the first admin from these variables):

```
export KC_BOOTSTRAP_ADMIN_USERNAME=temp-admin
export KC_BOOTSTRAP_ADMIN_PASSWORD=*****
bin/kc.sh start-dev --http-port=8080
```

4. Open `http://localhost:8080/admin` and sign in as the temporary admin.
5. Create a **permanent, named administrator** `iamadmin` in the `master` realm (Users → Add user). Set its password on the Credentials tab and give it the `admin` realm role on the Role mapping tab.
6. Sign in again as `iamadmin` and **delete** ``temp-admin``. Keycloak marks the bootstrap account as temporary, and leaving a shared bootstrap credential in place is a security risk.

```

$ cat /etc/os-release | head -4
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"

$ uname -srm
Linux 6.18.44-fc-v37 x86_64

$ nproc && free -h | head -2
4
Mem:          total        used        free      shared  buff/cache   available
Mem:          15Gi         1.2Gi         13Gi         12Mi         1.4Gi         14Gi

$ java -version
openjdk version "21.0.10" 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)

$ ./bin/kc.sh --version
Keycloak 26.7.4
JVM: 21.0.10 (Ubuntu OpenJDK 64-Bit Server VM 21.0.10+7-Ubuntu-124.04)
OS: Linux 6.18.44-fc-v37 amd64

$ KC_BOOTSTRAP_ADMIN_USERNAME=temp-admin KC_BOOTSTRAP_ADMIN_PASSWORD=***** ./bin/kc.sh start-dev --http-port=8080
2026-09-28 19:34:47.799 INFO [org.keycloak.services] (main) KC-SERVICES0050: Initializing master realm
2026-09-28 19:34:49.562 INFO [org.keycloak.services] (main) KC-SERVICES0077: Created temporary admin user with username temp-admin
2026-09-28 19:34:49.855 INFO [io.quarkus] (main) Keycloak 26.7.4 on JVM (powered by Quarkus 3.33.3.2) started in 13.901s. Listening on: http://localhost:8080
2026-09-28 19:34:49.855 INFO [io.quarkus] (main) Profile dev activated.

$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/admin/master/console/
200

```

Figure 1: Environment: OS, Java and Keycloak version, and the start-up log showing Keycloak 26.7.4 listening on port 8080

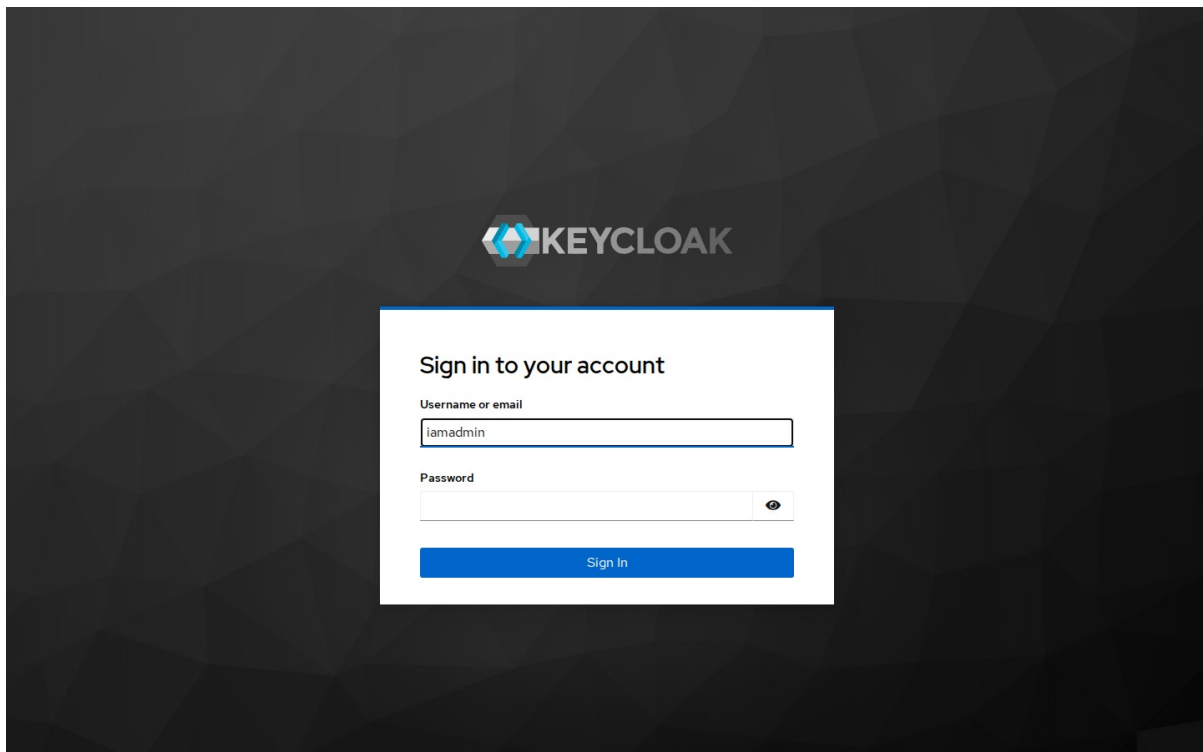


Figure 2: Keycloak Administration Console login page (master realm)

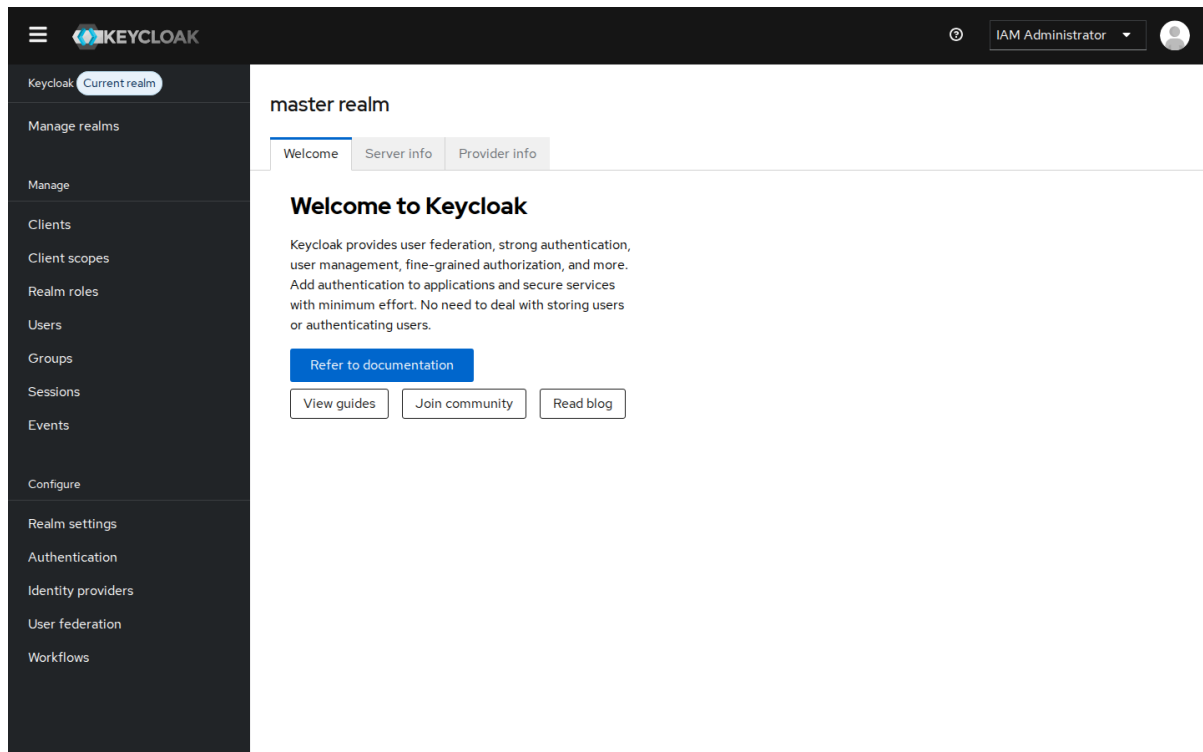


Figure 3: Successful access to the Administration Console as "IAM Administrator"

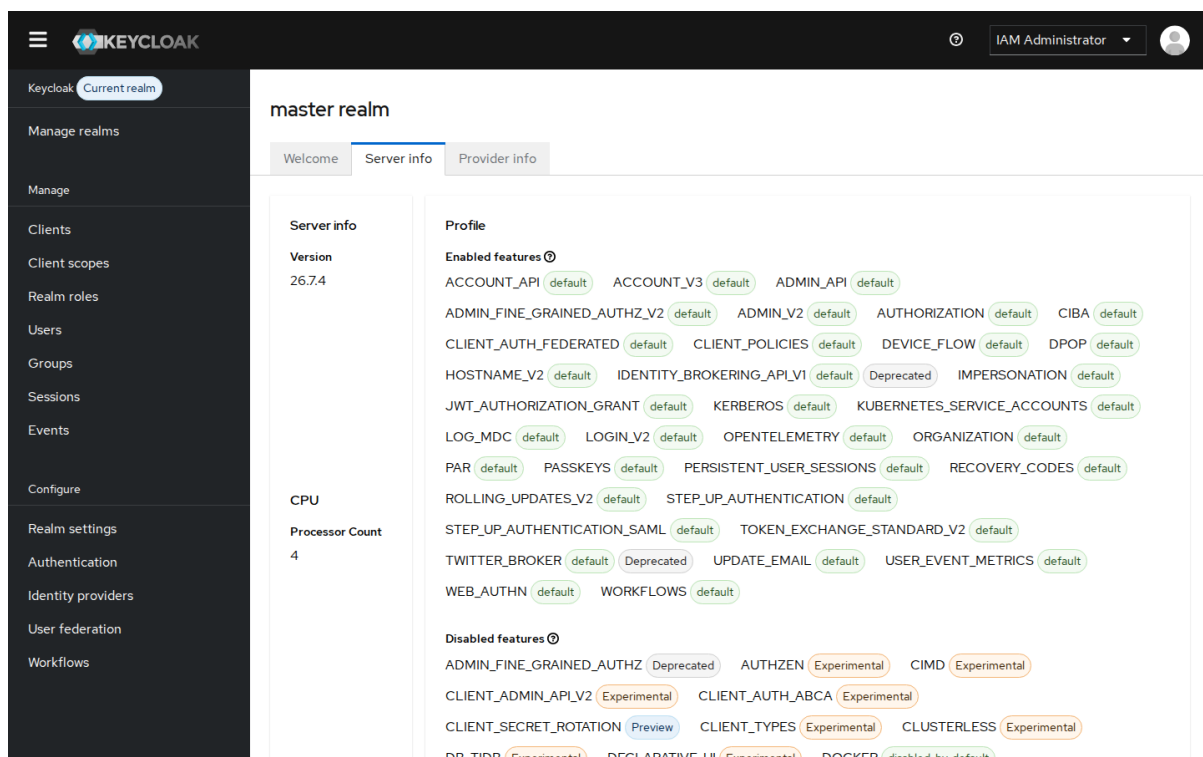


Figure 4: Server info tab confirming Keycloak version 26.7.4

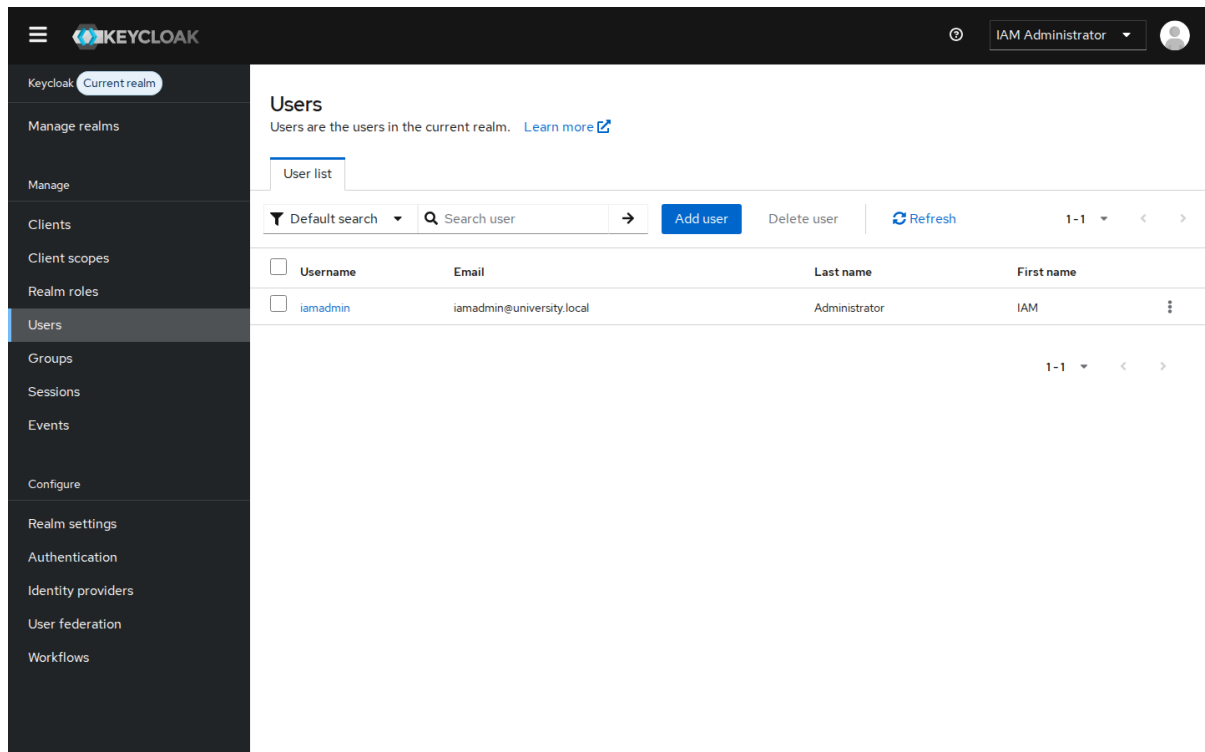


Figure 5: Permanent administrative account iamadmin in the master realm (temporary bootstrap admin removed)

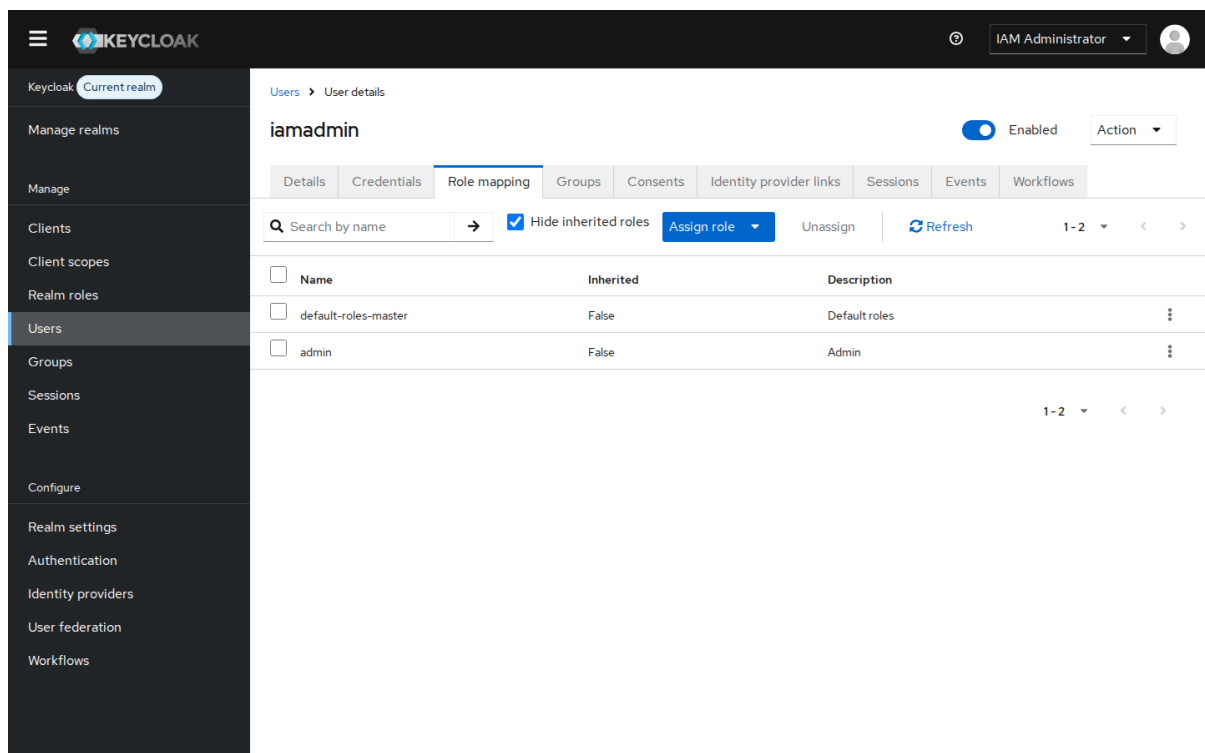


Figure 6: iamadmin holds the master-realm "admin" role

3. Realm Configuration (Task 2)

A realm is an isolated space that holds its own users, roles, clients and policies. The **master** realm is kept only for administering Keycloak. The institution's identities live in a separate realm, so ordinary users can never reach the Keycloak administration functions.

3.1 Creating UniversityRealm

Steps: open the realm selector and choose **Manage realms** → **Create realm**, enter Realm name **UniversityRealm**, and click Create. Then open Realm settings → General and set the Display name to "University Identity Service", which is shown on the login page.

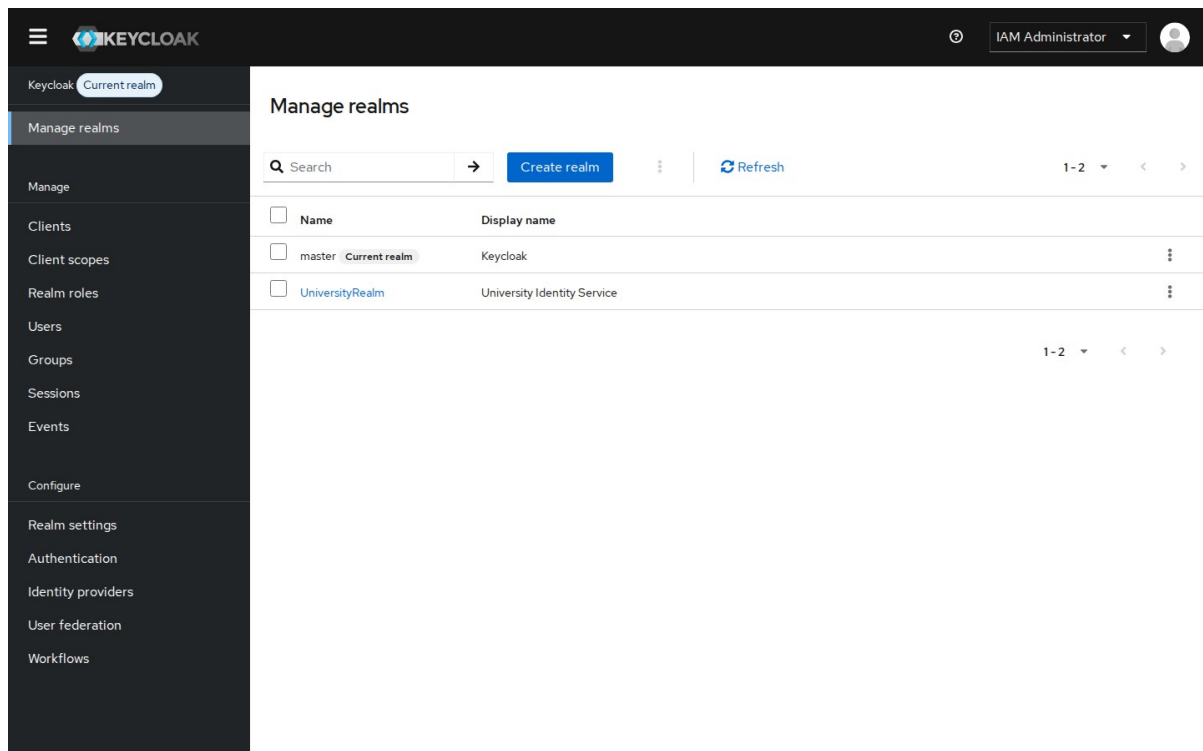


Figure 7: Manage realms: UniversityRealm created alongside master

University Identity Service Enabled Action

Realm settings are settings that control the options for users, applications, roles, and groups in the current realm. [Learn more](#)

General | Login | Email | Themes | Keys | Events | Localization | Security defenses | Sessions | Tokens | Clients

Real name * UniversityRealm

Display name University Identity Service

HTML Display name

Frontend URL

Require SSL External requests

ACR to LoA Mapping
No ACR to LoA Mapping have been defined yet. Click the below button to add ACR to LoA Mapping, key and value are required for a key pair.
[Add ACR to LoA Mapping](#)

User-managed access Off

Organizations Off

Admin Permissions Off

Unmanaged Attributes Disabled

Signature algorithm Choose...

SAML IdP metadata

[Save](#) [Revert](#)

Figure 8: UniversityRealm general settings (enabled, Require SSL: external requests)

3.2 Realm security settings

Setting	Value	Purpose
User registration	Off	Only the IAM administrator creates identities
Login with email	On; duplicate emails Off	Convenient and unique sign-in
Forgot password	On	Self-service reset through verified email
Password policy	min. 10 chars, 1 upper, 1 lower, 1 digit, 1 special, not username, history 3	Resist guessing and reuse
Brute-force detection	Lockout temporarily after 5 failures, wait 1 min increasing to 15 min	Stop password guessing
Token lifetimes	Access token 5 min, SSO idle 30 min, SSO max 10 h	Limit damage from stolen tokens or sessions
Events	Login and admin events saved for 30 days	Audit trail

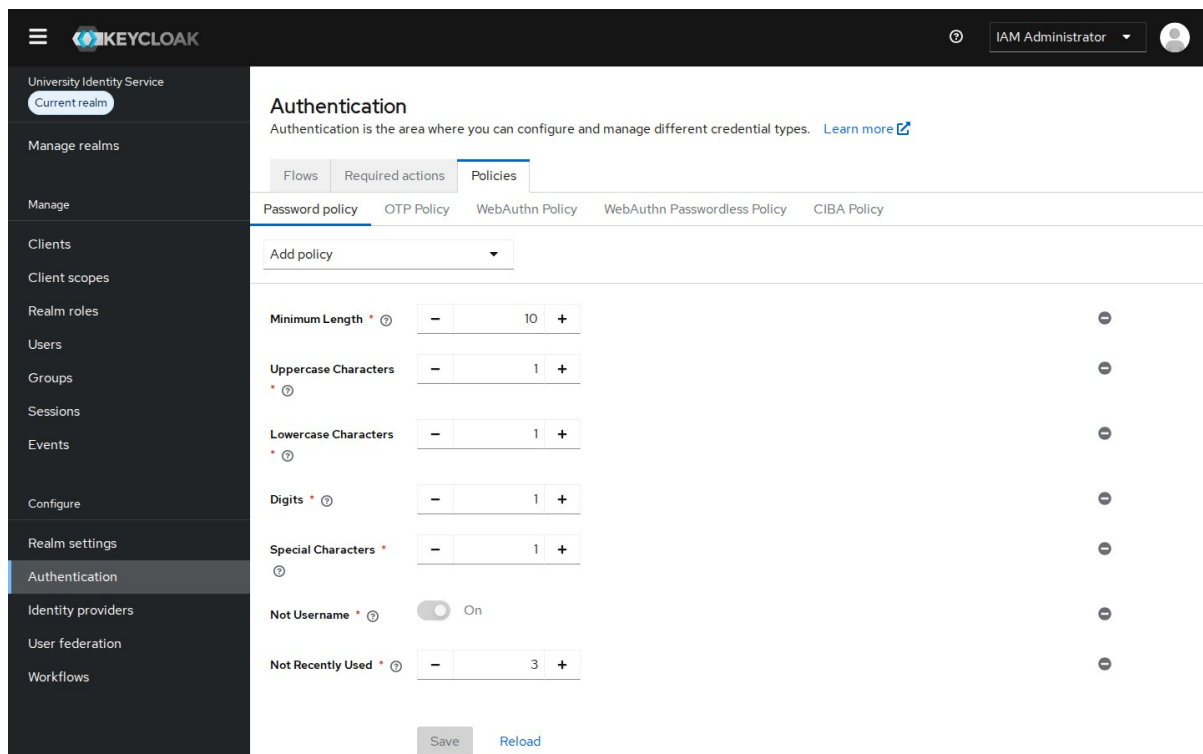


Figure 9: Authentication → Policies → Password policy for UniversityRealm

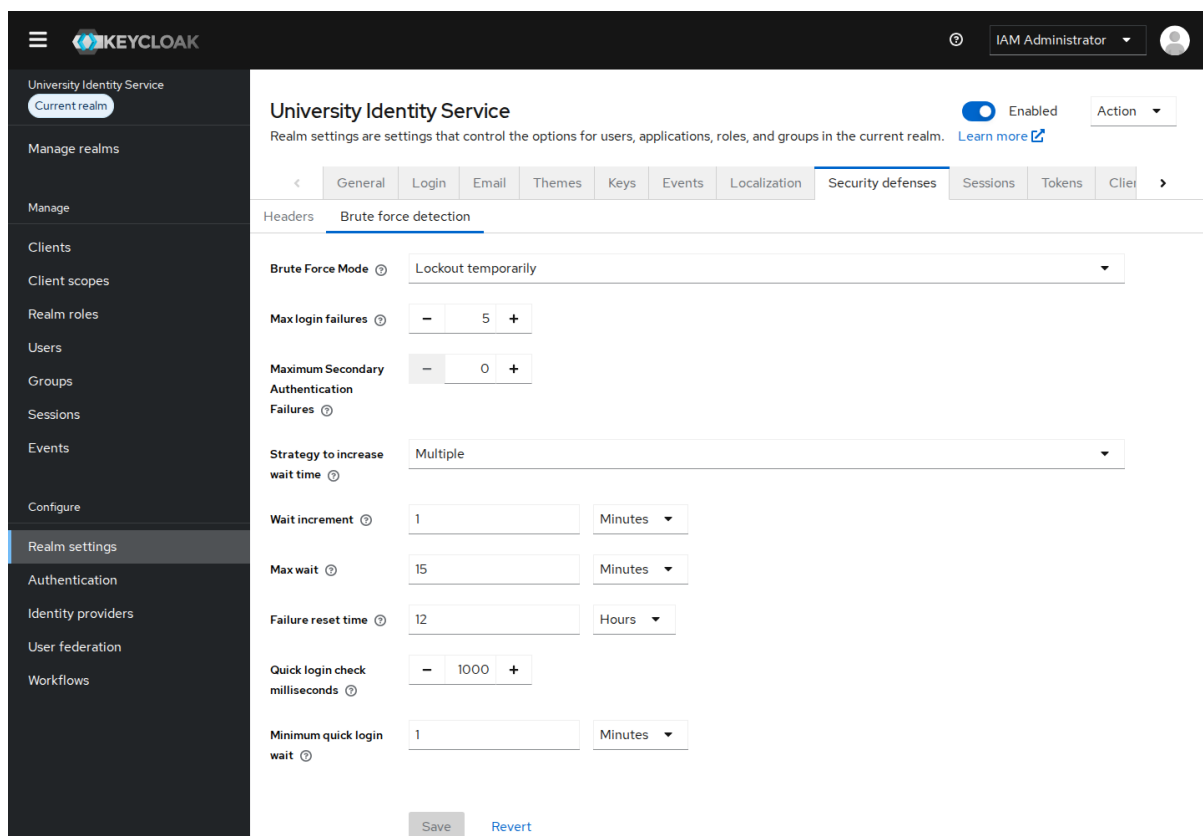


Figure 10: Realm settings → Security defenses → Brute force detection

4. User Configuration (Task 2)

Each user was created under Users → Add user with the required attributes. The password was then set on the **Credentials** tab with "Temporary" switched off, so it can be used for the demonstration.

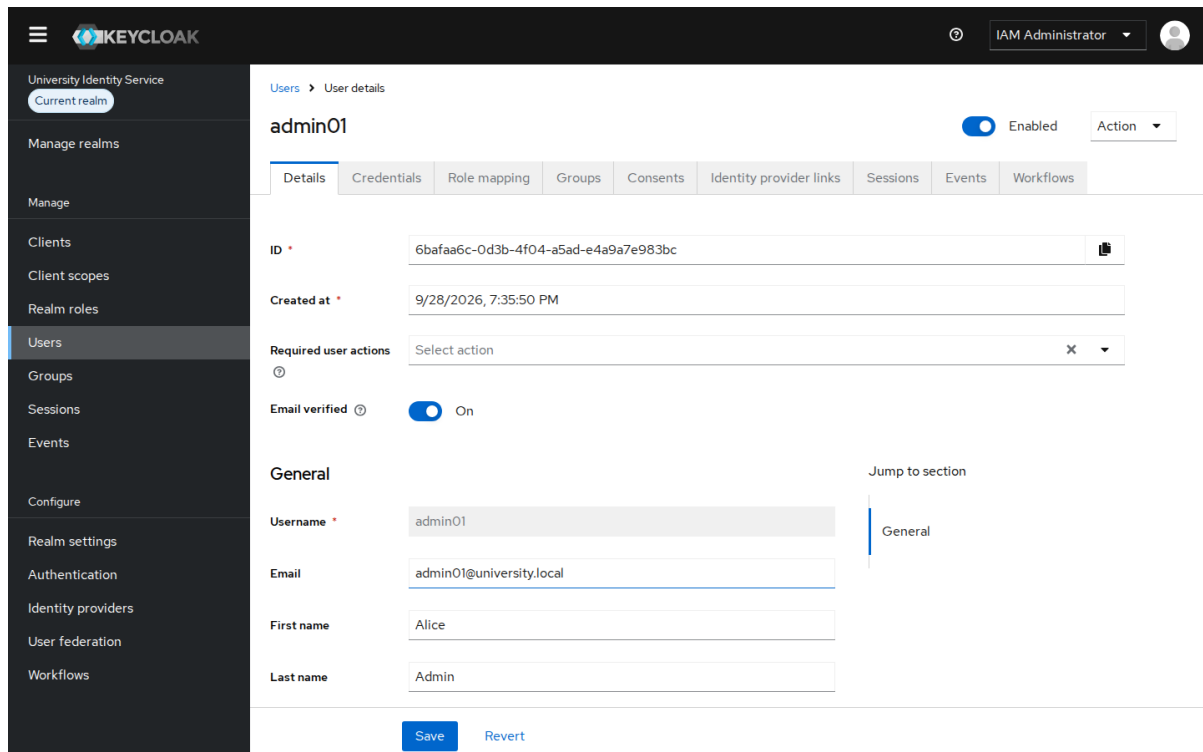
Username	First name	Last name	Email	Password
admin01	Alice	Admin	admin01@university.local	set (meets policy)
lecturer01	Leonard	Lecturer	lecturer01@university.local	set (meets policy)
student01	Stella	Student	student01@university.local	set (meets policy)

The screenshot shows the Keycloak administration console. The left sidebar contains the navigation menu with 'Users' selected. The main panel displays the 'Users' section with a 'User list' tab. Below the tab is a search bar and an 'Add user' button. A table lists the users:

☐	Username	Email	Last name	First name	
☐	admin01	admin01@university.local	Admin	Alice	⋮
☐	lecturer01	lecturer01@university.local	Lecturer	Leonard	⋮
☐	student01	student01@university.local	Student	Stella	⋮

At the bottom right of the table, there is a pagination control showing '1-3' and navigation arrows.

Figure 11: UniversityRealm users: admin01, lecturer01 and student01



The screenshot shows the Keycloak Admin Console interface. On the left is a dark sidebar with navigation links: University Identity Service, Current realm, Manage realms, Manage, Clients, Client scopes, Realm roles, Users (highlighted), Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, User federation, and Workflows. The main content area is titled 'Users > User details' and shows the details for user 'admin01'. At the top right, there is a toggle for 'Enabled' (turned on) and an 'Action' dropdown. Below this are tabs for Details, Credentials, Role mapping, Groups, Consents, Identity provider links, Sessions, Events, and Workflows. The 'Details' tab is active, showing fields for ID (6bafaa6c-0d3b-4f04-a5ad-e4a9a7e983bc), Created at (9/28/2026, 7:35:50 PM), Required user actions (Select action), and Email verified (On). The 'General' section contains fields for Username (admin01), Email (admin01@university.local), First name (Alice), and Last name (Admin). A 'Jump to section' dropdown is also present, currently set to 'General'. At the bottom are 'Save' and 'Revert' buttons.

University Identity Service
Current realm

Manage realms

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Workflows

Users > User details

admin01

Enabled Action

Details Credentials Role mapping Groups Consents Identity provider links Sessions Events Workflows

ID * 6bafaa6c-0d3b-4f04-a5ad-e4a9a7e983bc

Created at * 9/28/2026, 7:35:50 PM

Required user actions Select action X

Email verified On

General

Username * admin01

Email admin01@university.local

First name Alice

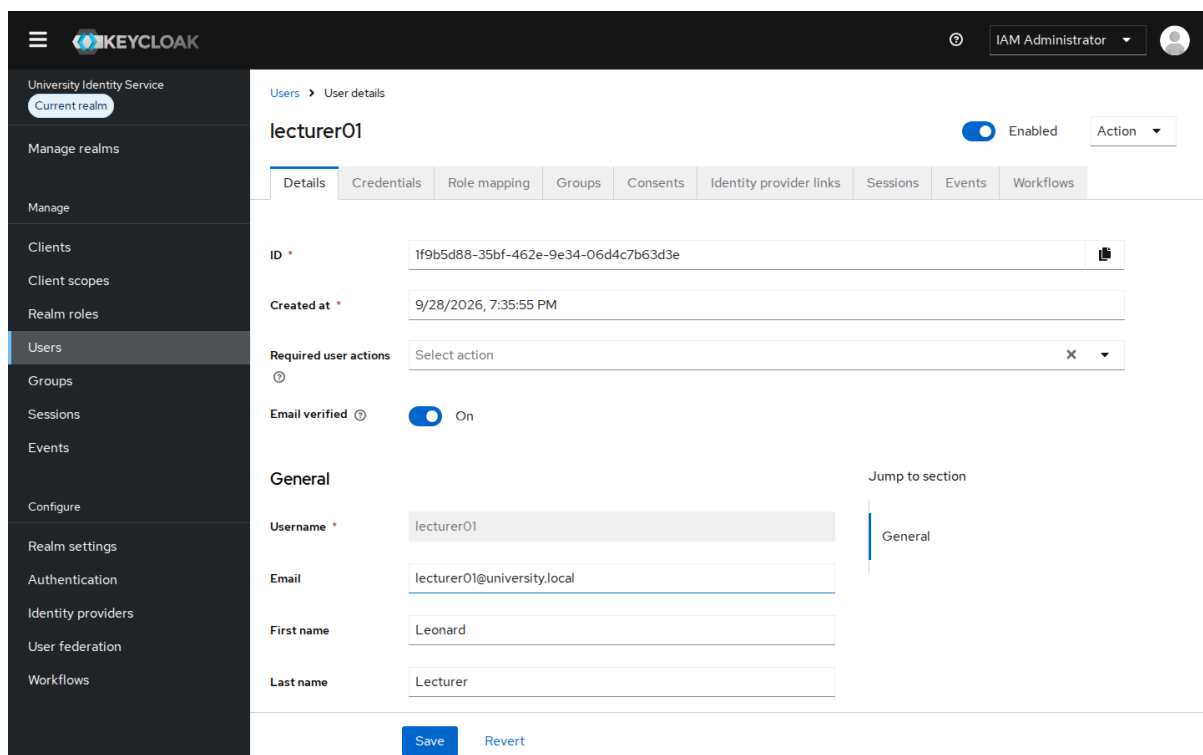
Last name Admin

Jump to section

General

Save Revert

Figure 12: admin01: username, email, first name, last name



The screenshot shows the Keycloak Admin Console interface for user 'lecturer01'. The sidebar is identical to the previous figure. The main content area shows 'Users > User details' for 'lecturer01'. The 'Enabled' toggle is turned on. The 'Details' tab is active, showing ID (1f9b5d88-35bf-462e-9e34-06d4c7b63d3e), Created at (9/28/2026, 7:35:55 PM), Required user actions (Select action), and Email verified (On). The 'General' section contains fields for Username (lecturer01), Email (lecturer01@university.local), First name (Leonard), and Last name (Lecturer). A 'Jump to section' dropdown is set to 'General'. At the bottom are 'Save' and 'Revert' buttons.

University Identity Service
Current realm

Manage realms

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Workflows

Users > User details

lecturer01

Enabled Action

Details Credentials Role mapping Groups Consents Identity provider links Sessions Events Workflows

ID * 1f9b5d88-35bf-462e-9e34-06d4c7b63d3e

Created at * 9/28/2026, 7:35:55 PM

Required user actions Select action X

Email verified On

General

Username * lecturer01

Email lecturer01@university.local

First name Leonard

Last name Lecturer

Jump to section

General

Save Revert

Figure 13: lecturer01 user details

The screenshot shows the Keycloak user details page for 'student01'. The left sidebar contains navigation links: University Identity Service, Current realm, Manage realms, Manage, Clients, Client scopes, Realm roles, Users (selected), Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, User federation, and Workflows. The main content area has a breadcrumb 'Users > User details' and a toggle for 'Enabled' (checked). Below this are tabs: Details (selected), Credentials, Role mapping, Groups, Consents, Identity provider links, Sessions, Events, and Workflows. The 'Details' tab shows fields for ID (1259064b-403b-4f22-b50c-bc43b930abc2), Created at (9/28/2026, 7:36:00 PM), Required user actions (Select action), and Email verified (On). The 'General' section includes Username (student01), Email (student01@university.local), First name (Stella), and Last name (Student). A 'Jump to section' dropdown is set to 'General'. At the bottom are 'Save' and 'Revert' buttons.

Figure 14: student01 user details

The screenshot shows the Keycloak user details page for 'student01', with the 'Credentials' tab selected. The left sidebar is identical to Figure 14. The main content area shows the 'Credentials' tab with a 'Credential Reset' button. Below this is a table of credentials:

Type	User label	Created at	Data
Password		9/28/2026, 7:36:02 PM	Show data

Below the table are buttons for 'Reset password' and a vertical ellipsis menu.

Figure 15: Credentials tab: a password credential is stored (hashed) for student01

4.1 Enabling and disabling a user account

Demonstration: on student01's Details page I switched the **Enabled** toggle off and confirmed the dialog. The Users list then shows a "Disabled" badge. When student01 tries to sign in to the Student Portal, Keycloak rejects the attempt with

"Account is disabled, contact your administrator." The account was then switched back on and login worked again.

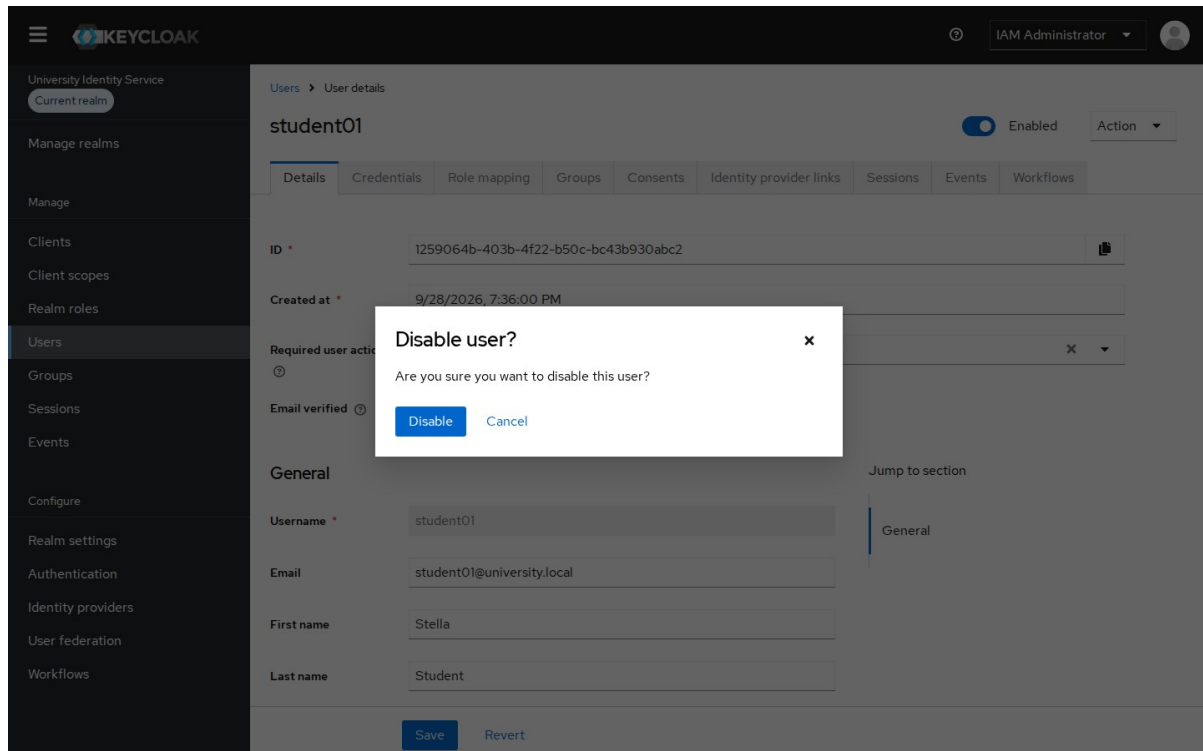


Figure 16: Disabling student01: confirmation dialog

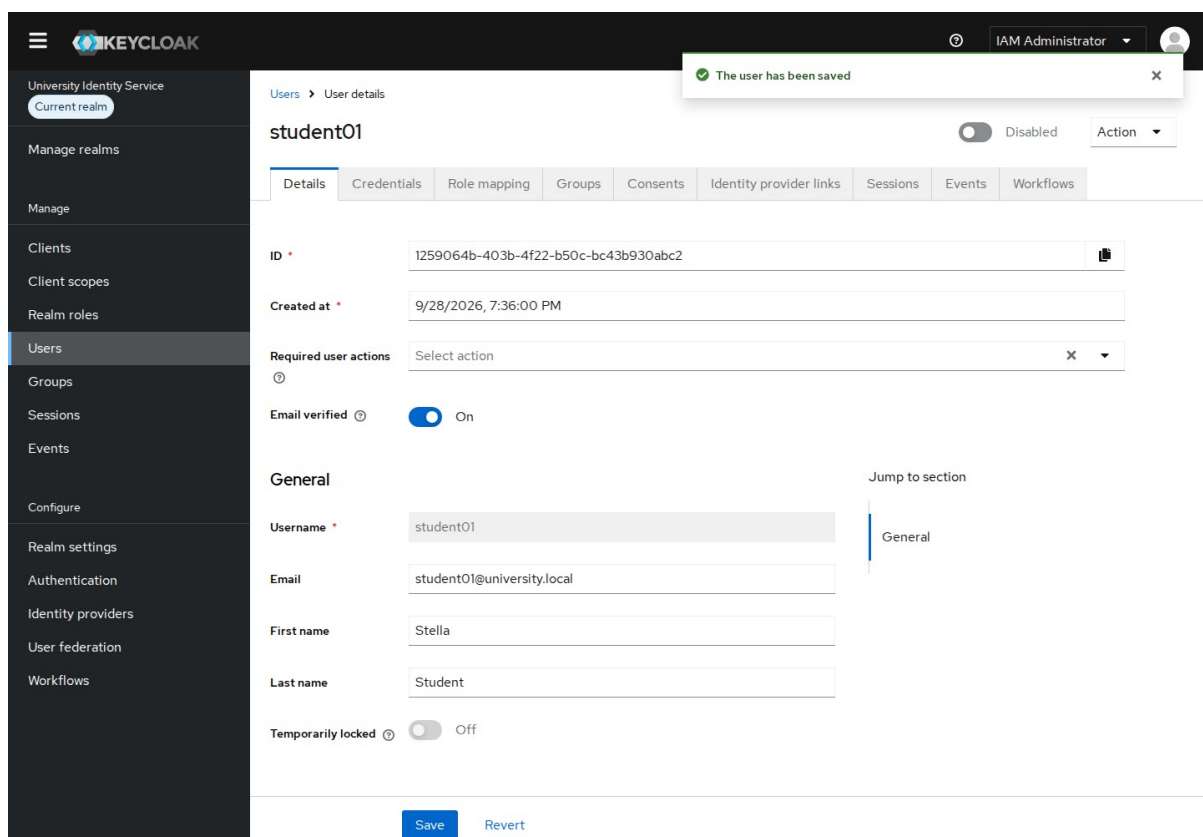


Figure 17: student01 saved as Disabled

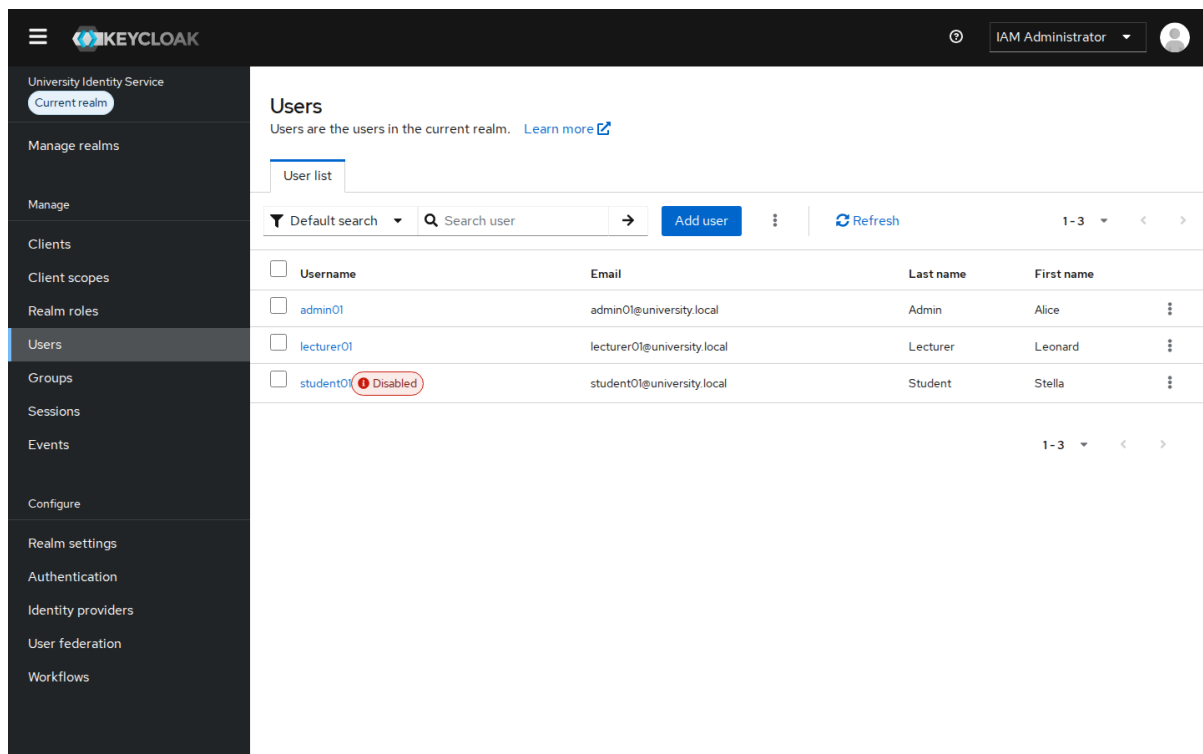


Figure 18: Users list showing student01 flagged as Disabled

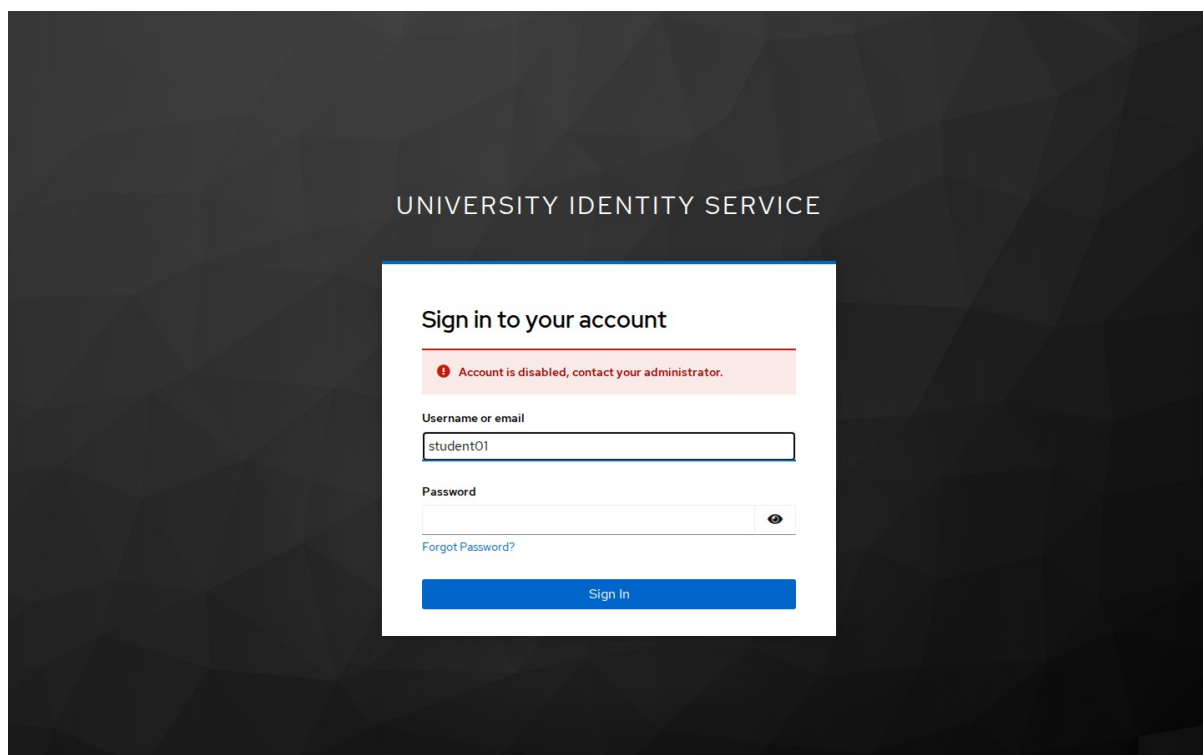


Figure 19: Login attempt by the disabled account is refused by Keycloak

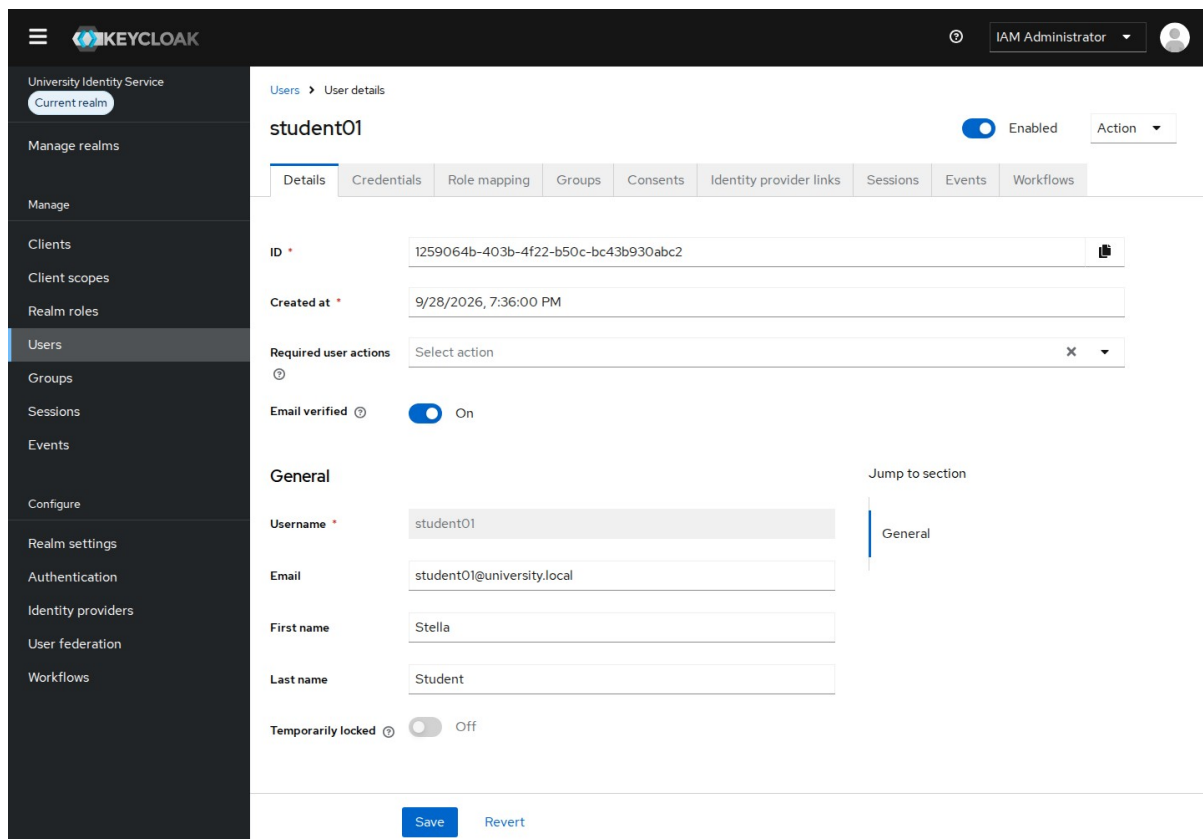


Figure 20: *student01 re-enabled*

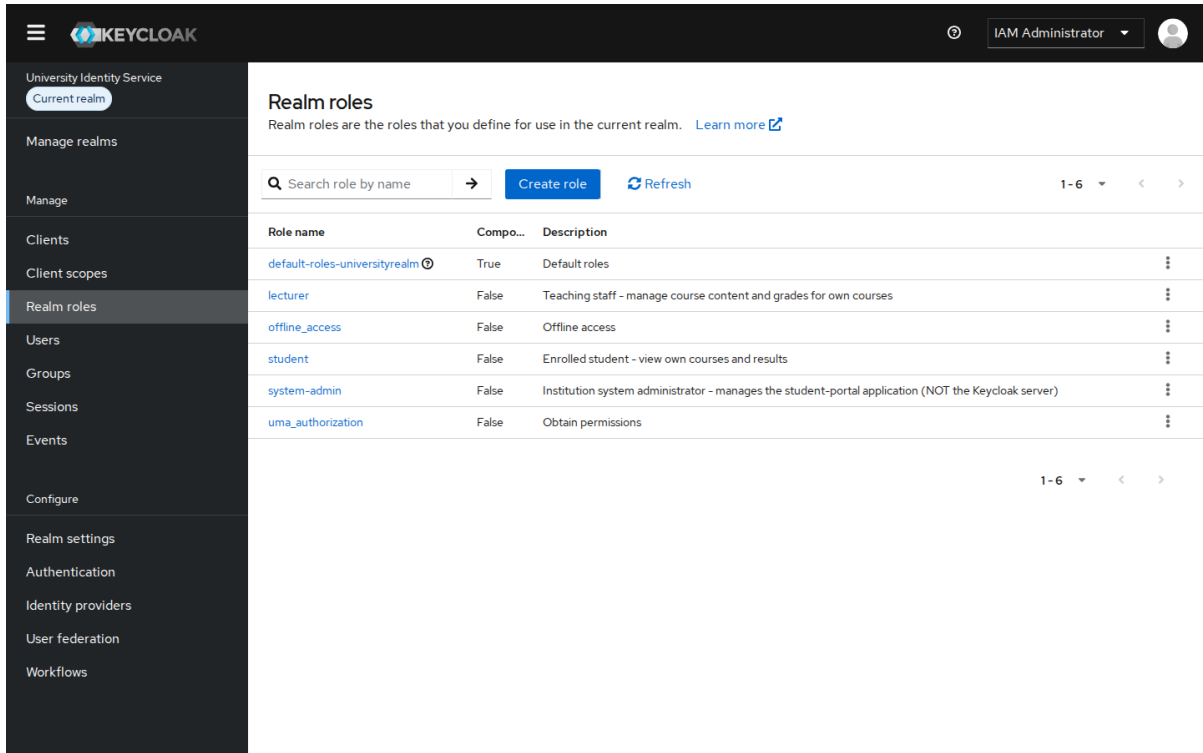
When this control is needed. Disabling blocks authentication immediately but keeps the identity, its roles and its audit history. Deleting would lose all of those. Typical cases are:

- a student is suspended, has deferred, or has not cleared fees;
- a staff member is on extended leave, or their contract has ended and offboarding is still pending;
- **suspected compromise:** stolen credentials or unusual login events. The account is frozen while the incident is investigated;
- graduates and leavers are disabled first and deleted later under the records-retention policy;
- a disciplinary or legal hold, where evidence must be preserved.

5. Role Configuration (Task 3)

5.1 Creating the roles

Under Realm roles → Create role I created three **realm roles**, each with a description. Realm roles, rather than client roles, were chosen so that any future institutional application such as an LMS or library system can reuse the same roles.



Keycloak Interface: Realm roles

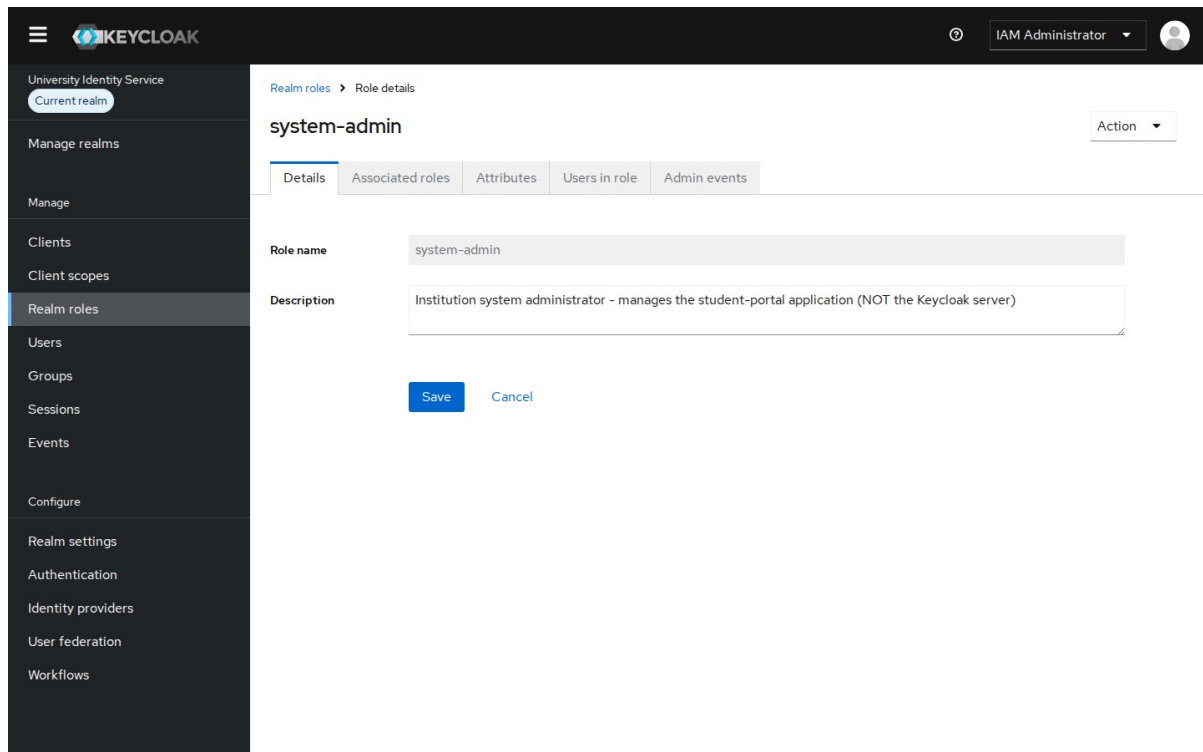
Realm roles are the roles that you define for use in the current realm. [Learn more](#)

Search role by name → [Create role](#) [Refresh](#) 1-6

Role name	Compo...	Description
default-roles-universityrealm	True	Default roles
lecturer	False	Teaching staff - manage course content and grades for own courses
offline_access	False	Offline access
student	False	Enrolled student - view own courses and results
system-admin	False	Institution system administrator - manages the student-portal application (NOT the Keycloak server)
uma_authorization	False	Obtain permissions

1-6

Figure 21: Realm roles: system-admin, lecturer and student (plus Keycloak defaults)



The screenshot shows the Keycloak administration interface. On the left is a dark sidebar with a menu. The top of the sidebar has the Keycloak logo and 'University Identity Service'. Below that is a 'Current realm' button. The menu items are: Manage realms, Manage, Clients, Client scopes, Realm roles (highlighted), Users, Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, User federation, and Workflows. The main content area has a top bar with 'IAM Administrator' and a user icon. Below the top bar, the breadcrumb is 'Realm roles > Role details'. The role name 'system-admin' is displayed at the top right of the main area. There are five tabs: Details (active), Associated roles, Attributes, Users in role, and Admin events. The 'Details' tab contains a form with 'Role name' (system-admin) and 'Description' (Institution system administrator - manages the student-portal application (NOT the Keycloak server)). At the bottom of the form are 'Save' and 'Cancel' buttons.

University Identity Service
Current realm

Manage realms

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Workflows

Realm roles > Role details

system-admin

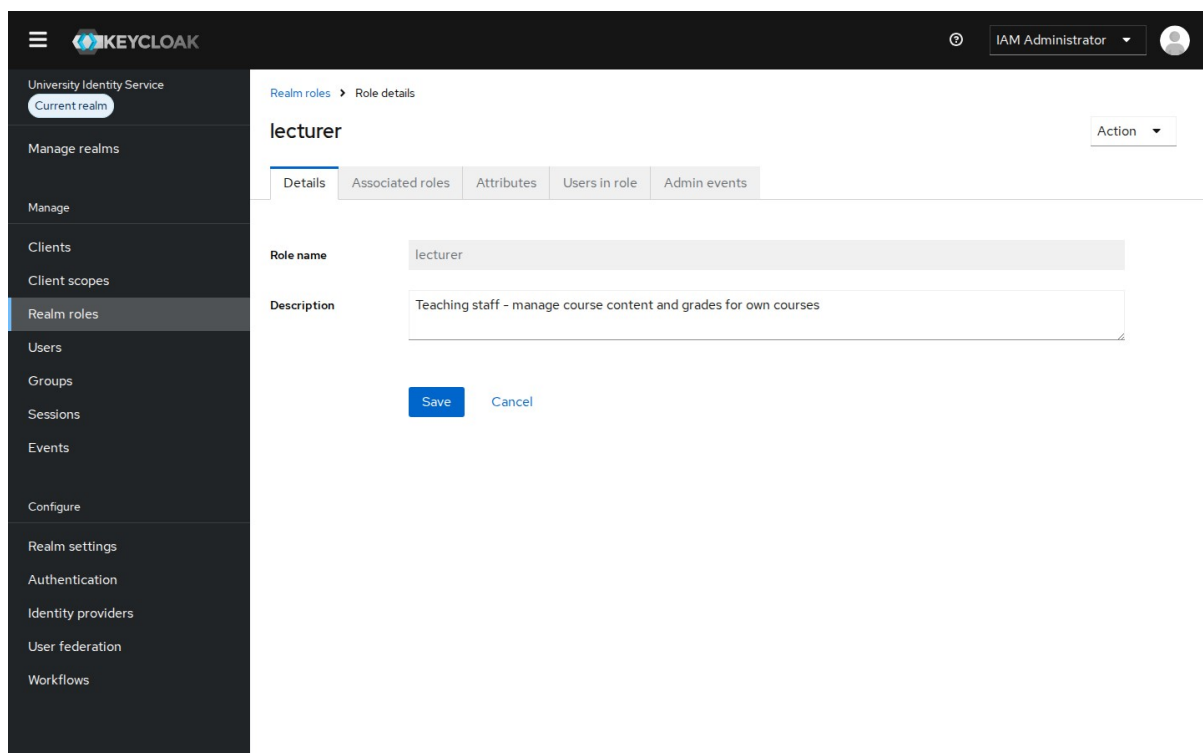
Action

Details Associated roles Attributes Users in role Admin events

Role name system-admin

Description Institution system administrator - manages the student-portal application (NOT the Keycloak server)

Save Cancel

Figure 22: Role details: system-admin

The screenshot shows the Keycloak administration interface for the 'lecturer' role. The sidebar is identical to the previous figure. The main content area has a top bar with 'IAM Administrator' and a user icon. The breadcrumb is 'Realm roles > Role details'. The role name 'lecturer' is displayed at the top right. The 'Details' tab is active, showing a form with 'Role name' (lecturer) and 'Description' (Teaching staff - manage course content and grades for own courses). 'Save' and 'Cancel' buttons are at the bottom.

University Identity Service
Current realm

Manage realms

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Workflows

Realm roles > Role details

lecturer

Action

Details Associated roles Attributes Users in role Admin events

Role name lecturer

Description Teaching staff - manage course content and grades for own courses

Save Cancel

Figure 23: Role details: lecturer

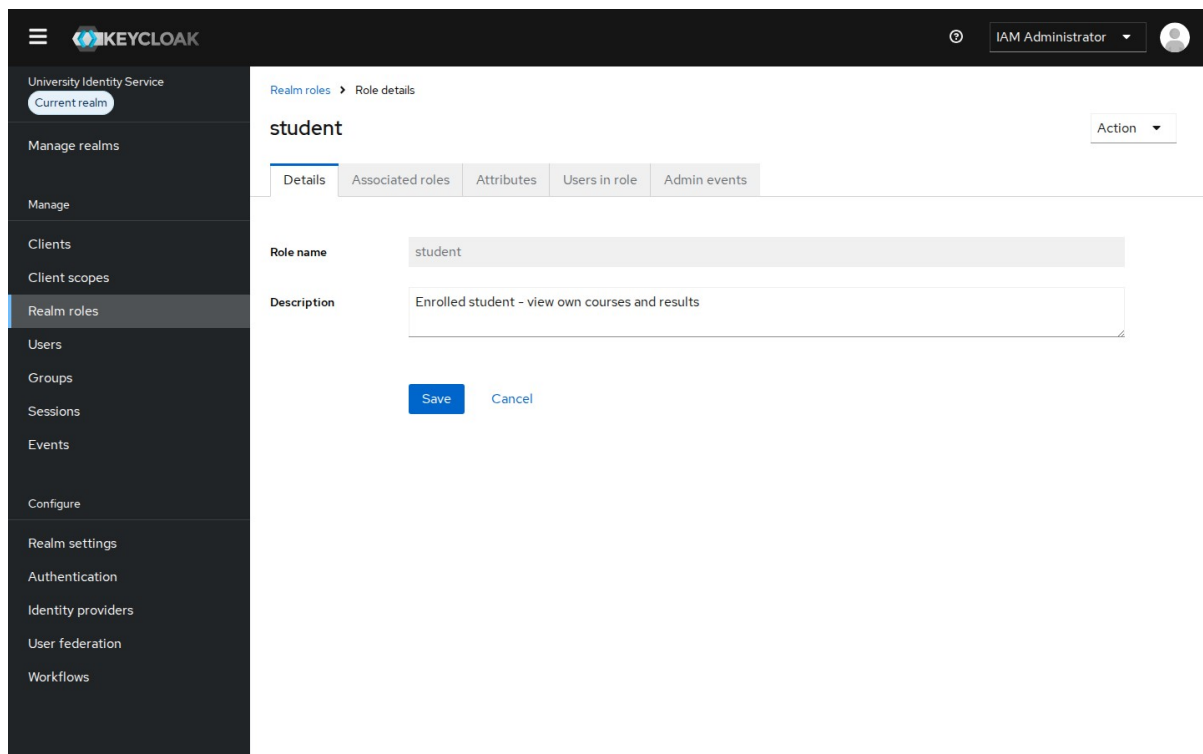


Figure 24: Role details: student

5.2 Assigning roles to users

Roles were assigned from each user's **Role mapping** → **Assign role** tab (filter by realm roles):

User	Assigned role	Access in student-portal
admin01	system-admin	/admin (System Administration)
lecturer01	lecturer	/lecturer/grades (Grade Management)
student01	student	/student/courses (My Courses & Results)

The screenshot shows the Keycloak Admin Console interface. The left sidebar contains navigation options: University Identity Service, Manage realms, Manage, Clients, Client scopes, Realm roles, Users (selected), Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, User federation, and Workflows. The main content area is titled 'Users > User details' and shows the user 'admin01' with a status of 'Enabled'. Below this, there are tabs for Details, Credentials, Role mapping (selected), Groups, Consents, Identity provider links, Sessions, Events, and Workflows. A search bar and a 'Hide inherited roles' checkbox (checked) are present. A table lists the assigned roles:

☐	Name	Inheri...	Description
☐	default-roles-universityrealm	False	Default roles
☐	system-admin	False	Institution system administrator - manages the student-portal application (NOT the Keycloak server)

At the bottom right of the table, there is a pagination indicator '1-2' and navigation arrows.

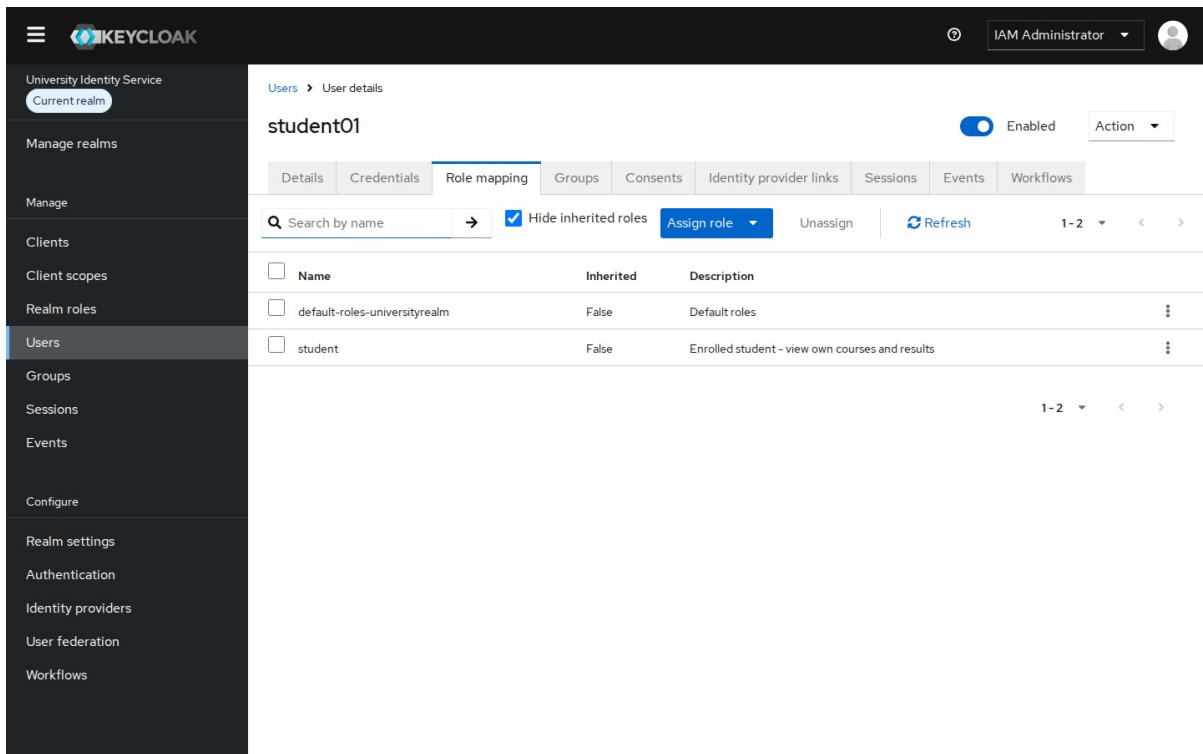
Figure 25: *admin01* → *system-admin*

The screenshot shows the Keycloak Admin Console interface. The left sidebar contains navigation options: University Identity Service, Manage realms, Manage, Clients, Client scopes, Realm roles, Users (selected), Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, User federation, and Workflows. The main content area is titled 'Users > User details' and shows the user 'lecturer01' with a status of 'Enabled'. Below this, there are tabs for Details, Credentials, Role mapping (selected), Groups, Consents, Identity provider links, Sessions, Events, and Workflows. A search bar and a 'Hide inherited roles' checkbox (checked) are present. A table lists the assigned roles:

☐	Name	Inherited	Description
☐	lecturer	False	Teaching staff - manage course content and grades for own courses
☐	default-roles-universityrealm	False	Default roles

At the bottom right of the table, there is a pagination indicator '1-2' and navigation arrows.

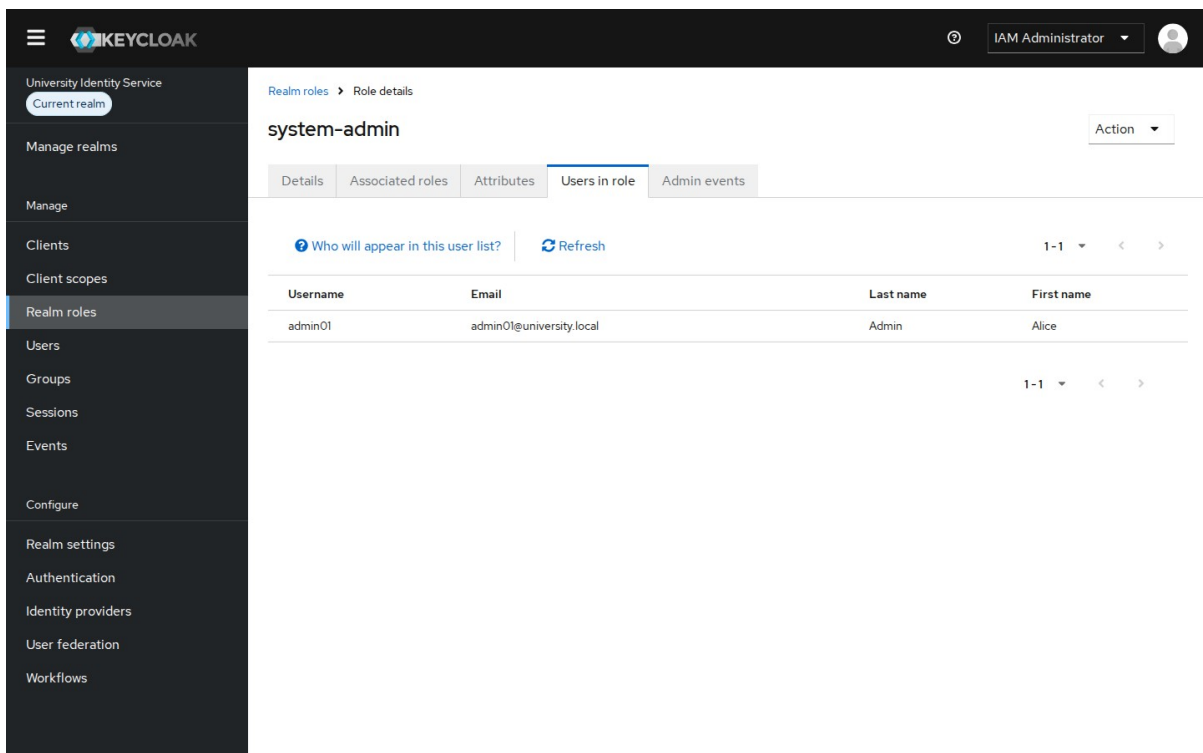
Figure 26: *lecturer01* → *lecturer*



The screenshot shows the Keycloak administration console. The left sidebar contains navigation links for 'University Identity Service', 'Manage realms', 'Manage', 'Clients', 'Client scopes', 'Realm roles', 'Users', 'Groups', 'Sessions', 'Events', 'Configure', 'Realm settings', 'Authentication', 'Identity providers', 'User federation', and 'Workflows'. The 'Users' link is selected. The main content area is titled 'Users > User details' and shows the details for 'student01'. The user is 'Enabled'. Below the tabs (Details, Credentials, Role mapping, Groups, Consents, Identity provider links, Sessions, Events, Workflows), there is a search bar and a 'Hide inherited roles' checkbox which is checked. A table lists the roles assigned to the user:

☐	Name	Inherited	Description
☐	default-roles-universityrealm	False	Default roles
☐	student	False	Enrolled student - view own courses and results

Figure 27: *student01* → *student*



The screenshot shows the Keycloak administration console. The left sidebar is the same as in Figure 27, but the 'Realm roles' link is selected. The main content area is titled 'Realm roles > Role details' and shows the details for 'system-admin'. The 'Users in role' tab is selected, showing a list of users assigned to this role:

Username	Email	Last name	First name
admin01	admin01@university.local	Admin	Alice

Figure 28: "Users in role" for *system-admin*: only *admin01*

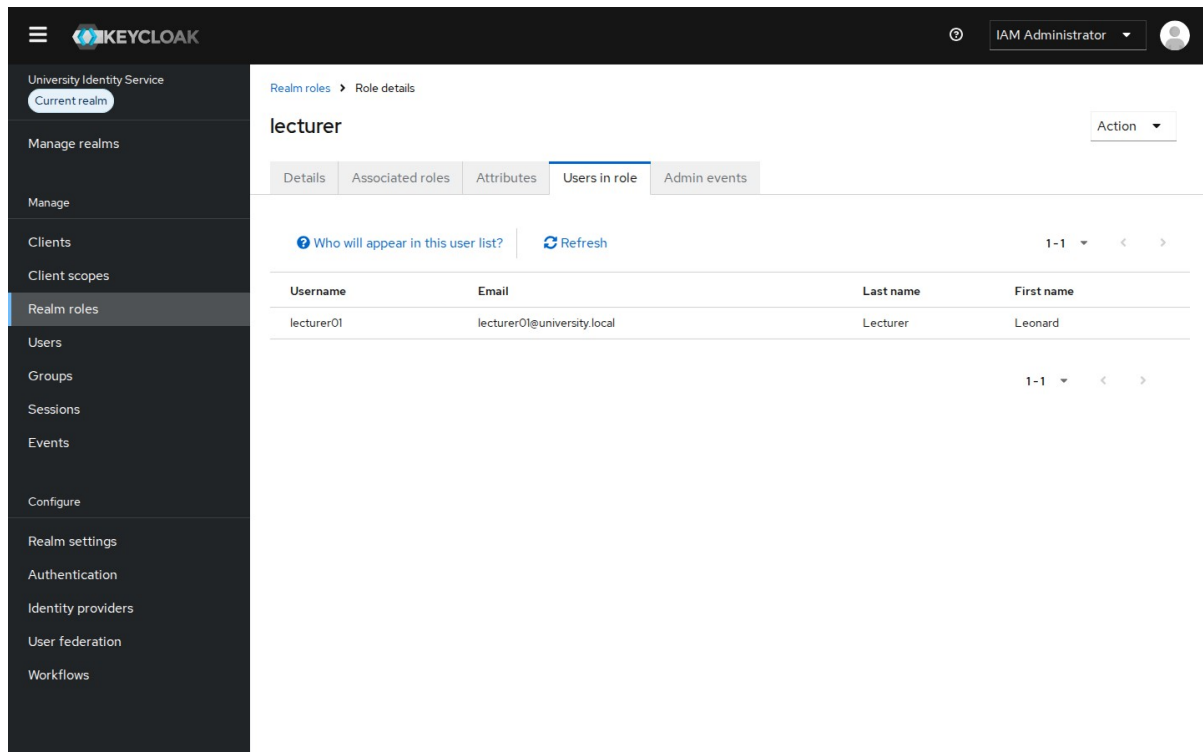


Figure 29: "Users in role" for lecturer: only lecturer01

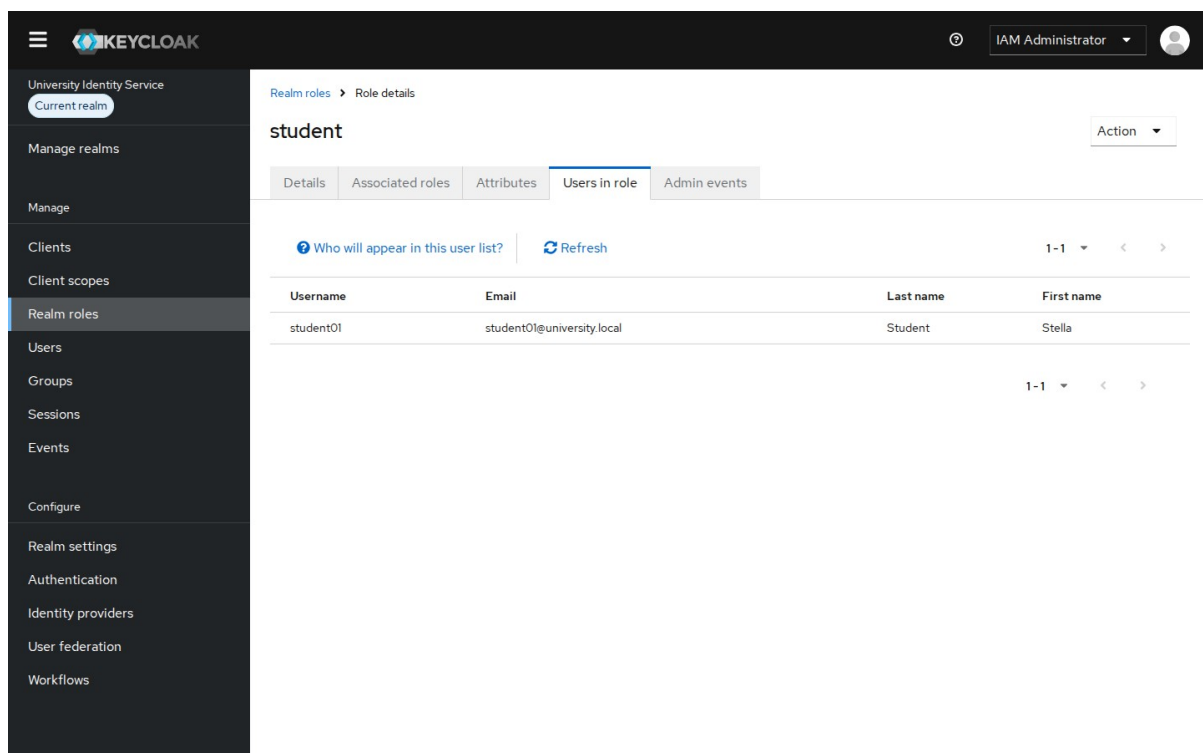


Figure 30: "Users in role" for student: only student01

5.3 How least privilege is applied

- **One role per user, no composite roles.** A user gets only the permissions their job needs. `system-admin` is deliberately **not** a composite of `lecturer` or `student`, so the administrator cannot read or change grades (demonstrated in section 5.4).

- **Application administrator is not the IdP administrator.** admin01 has **no realm-management** roles, so they cannot create users or change roles in Keycloak. Only **iamadmin** in the master realm can, which keeps IAM duties separate from application duties.
- **Deny by default in the application.** Every protected page checks for one specific role. If the role is missing, the application returns HTTP 403.
- **Minimal client capabilities.** student-portal has only the Standard (authorization code) flow. Implicit flow, direct password grants and service accounts are disabled.
- **Short-lived credentials.** Access tokens last 5 minutes and sessions expire after 30 idle minutes, so privileges that are removed or disabled take effect quickly.

5.4 Demonstrating different authorization privileges

Each user signed in to the Student Portal. The dashboard shows the roles Keycloak put in the token and which features are allowed. Each user was then sent directly to a page for another role, and the portal refused with **403 Access denied**.

Page (required role)	admin01	lecturer01	student01
/admin (system-admin)	ALLOWED	DENIED 403	DENIED
/lecturer/grades (lecturer)	DENIED 403	ALLOWED	DENIED 403
/student/courses (student)	DENIED	DENIED	ALLOWED

University Student Portal Signed in as **admin01** · [Token](#) · [Log out](#)

Authenticated via Keycloak

Username	admin01
Name	Alice Admin
Email	admin01@university.local
Subject (sub)	6bafaa6c-0d3b-4f04-a5ad-e4a9a7e983bc
Issuer	http://localhost:8080/realms/UniversityRealm
Realm roles (authorization)	system-admin

What you are authorized to do

ALLOWED

System Administration

Manage portal configuration, audit logs and service accounts.

Requires **system-admin**

[Open](#)

DENIED

Grade Management

Enter and publish marks for the courses you teach.

Requires **lecturer**

[Open](#)

DENIED

My Courses & Results

View your registered courses, timetable and released results.

Requires **student**

[Open](#)

Figure 31: admin01 dashboard: role system-admin, only System Administration allowed

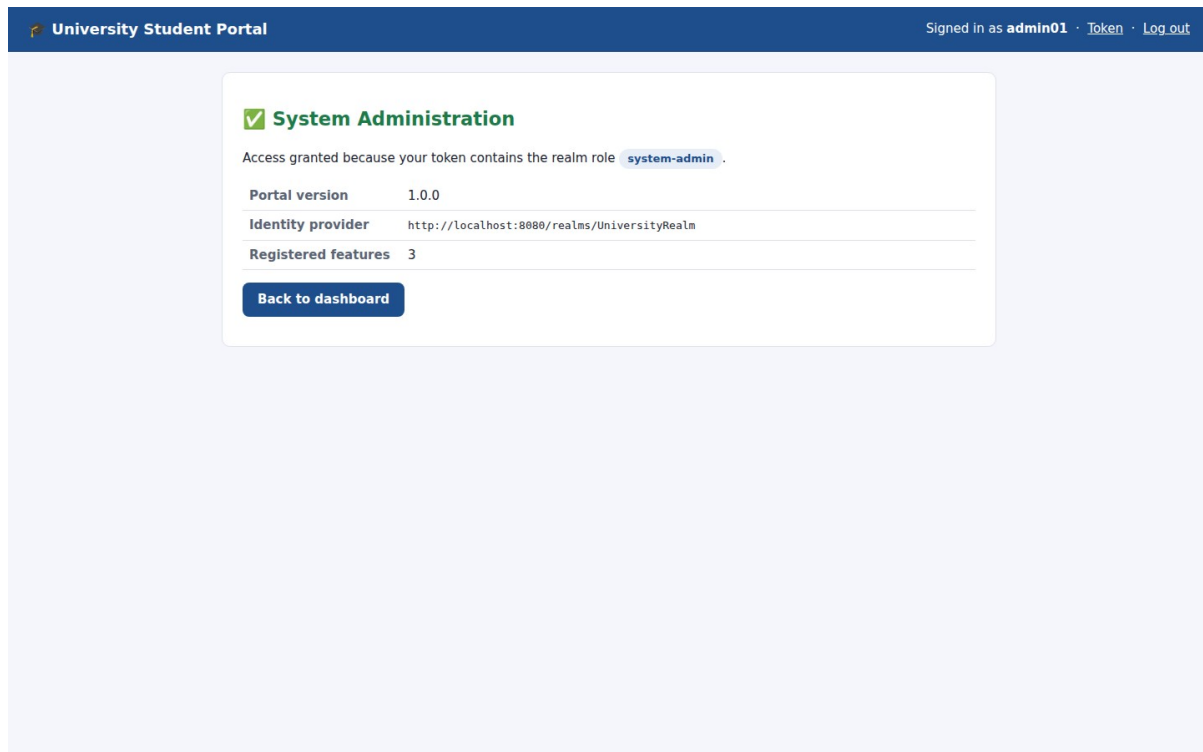


Figure 32: admin01 opens /admin: access granted

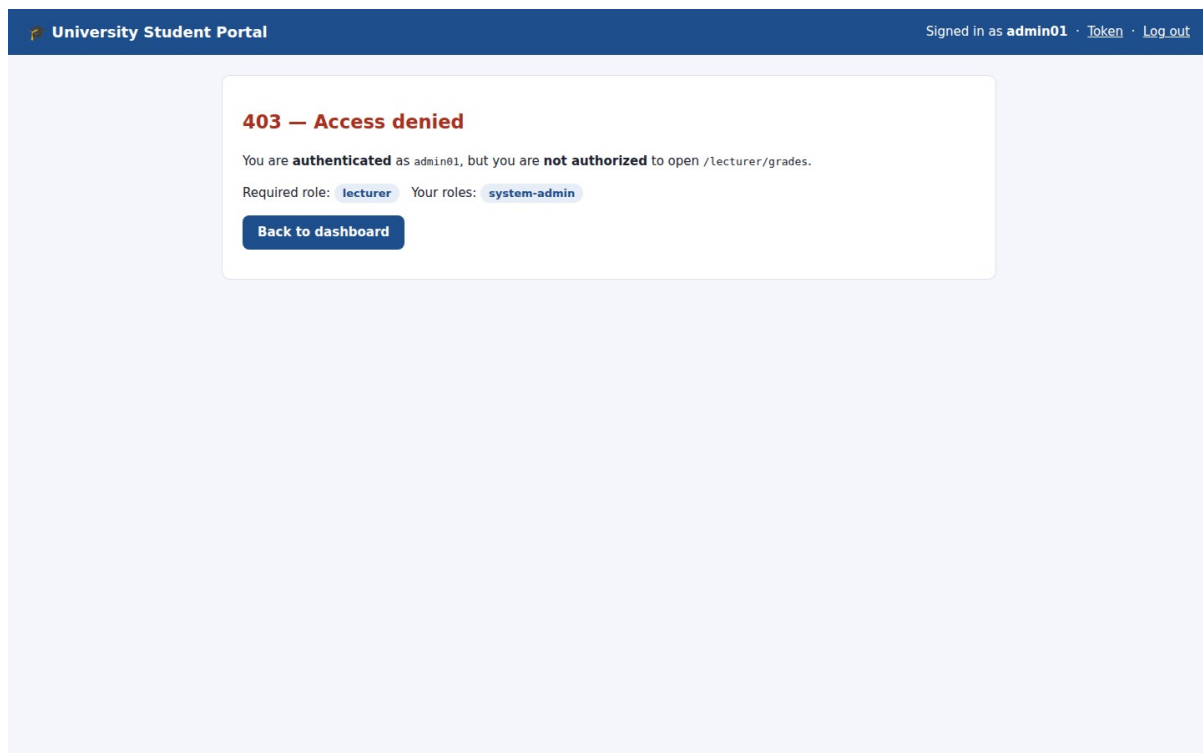


Figure 33: admin01 opens /lecturer/grades: 403, authenticated but not authorized

University Student Portal Signed in as **lecturer01** · [Token](#) · [Log out](#)

Authenticated via Keycloak

Username	lecturer01
Name	Leonard Lecturer
Email	lecturer01@university.local
Subject (sub)	1f9b5d88-35bf-462e-9e34-06d4c7b63d3e
Issuer	http://localhost:8080/realms/UniversityRealm
Realm roles (authorization)	lecturer

What you are authorized to do

DENIED

System Administration

Manage portal configuration, audit logs and service accounts.

Requires **system-admin**

[Open](#)

ALLOWED

Grade Management

Enter and publish marks for the courses you teach.

Requires **lecturer**

[Open](#)

DENIED

My Courses & Results

View your registered courses, timetable and released results.

Requires **student**

[Open](#)

Figure 34: lecturer01 dashboard: role lecturer, only Grade Management allowed

University Student Portal Signed in as **lecturer01** · [Token](#) · [Log out](#)

Grade Management

Access granted because your token contains the realm role **lecturer**.

Course	Students	Status
ISM 7101 Information Security Management	42	Draft marks
ISM 7105 Identity & Access Management	37	Published

[Back to dashboard](#)

Figure 35: lecturer01 opens /lecturer/grades: access granted

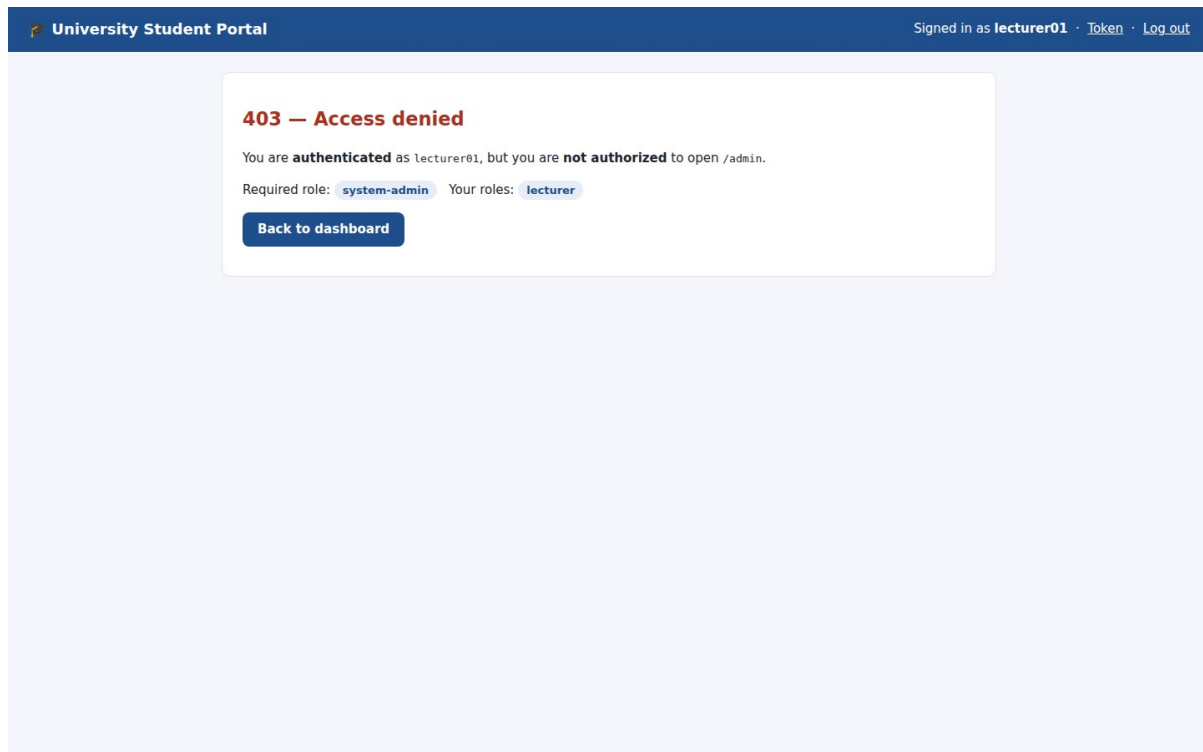


Figure 36: lecturer01 opens /admin: 403 denied

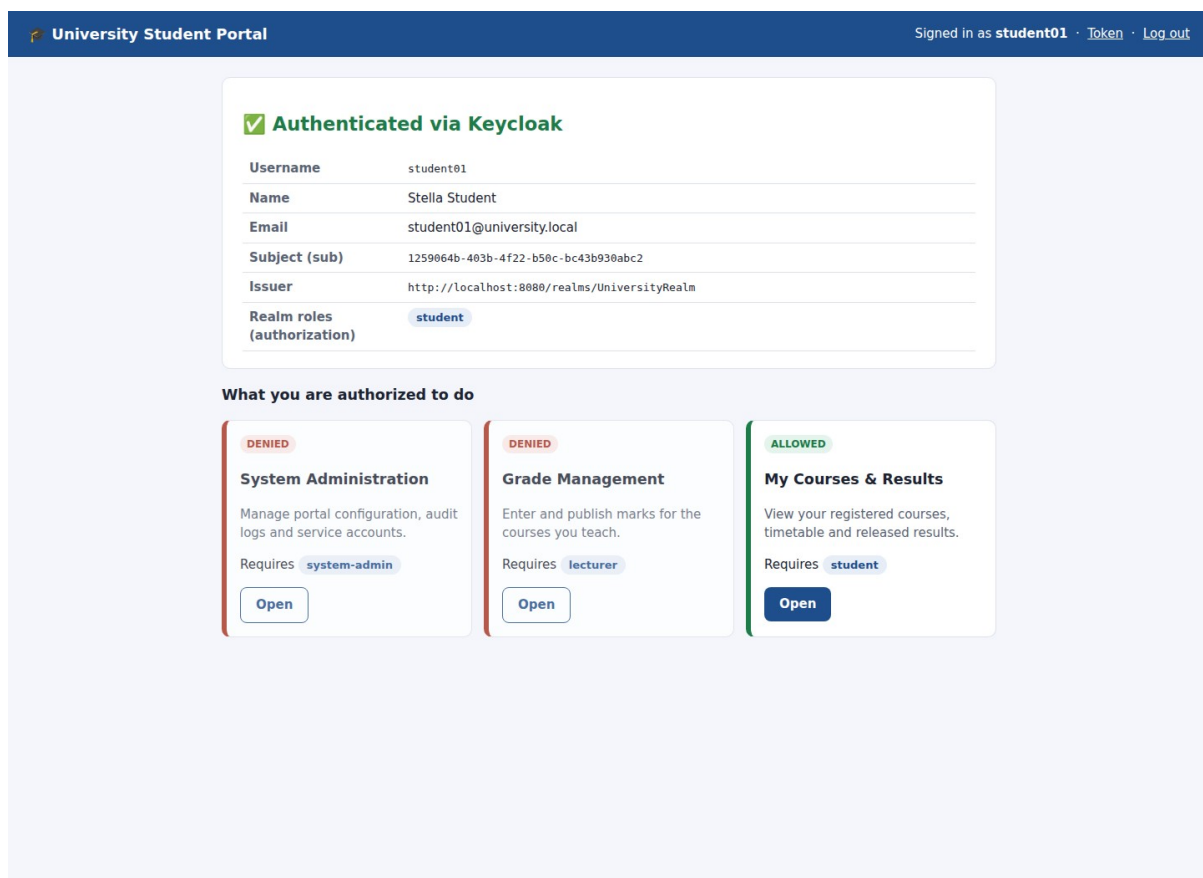
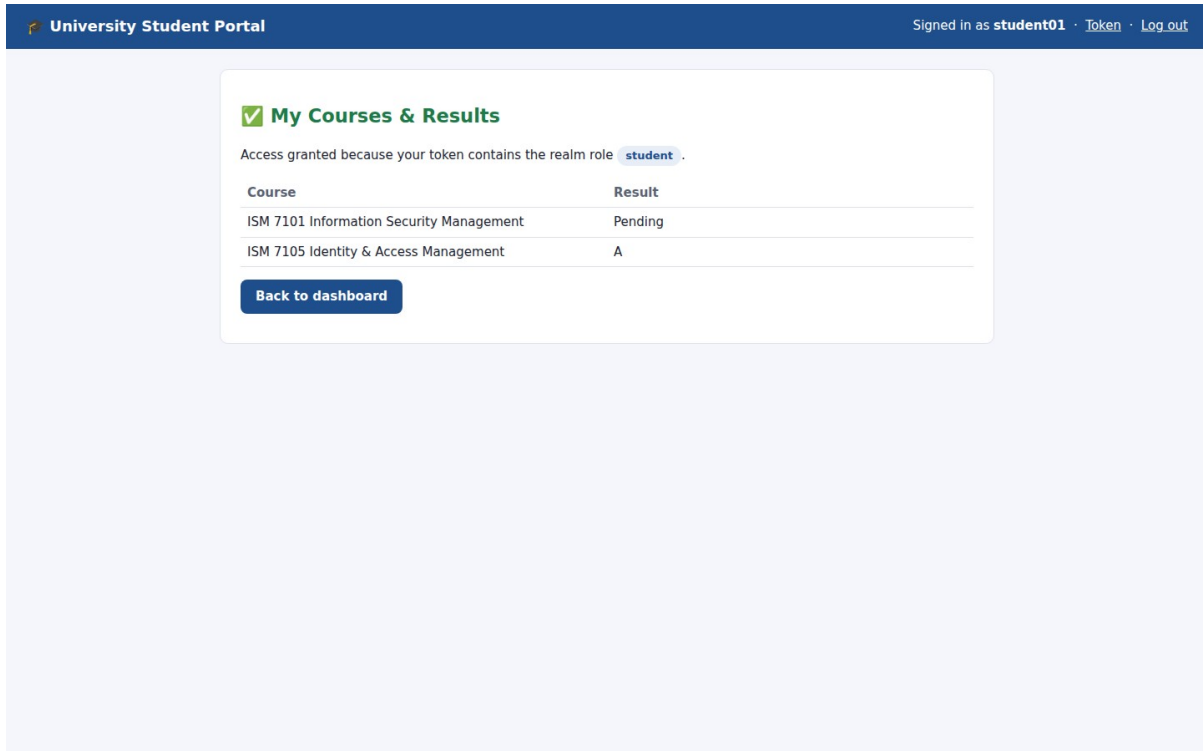


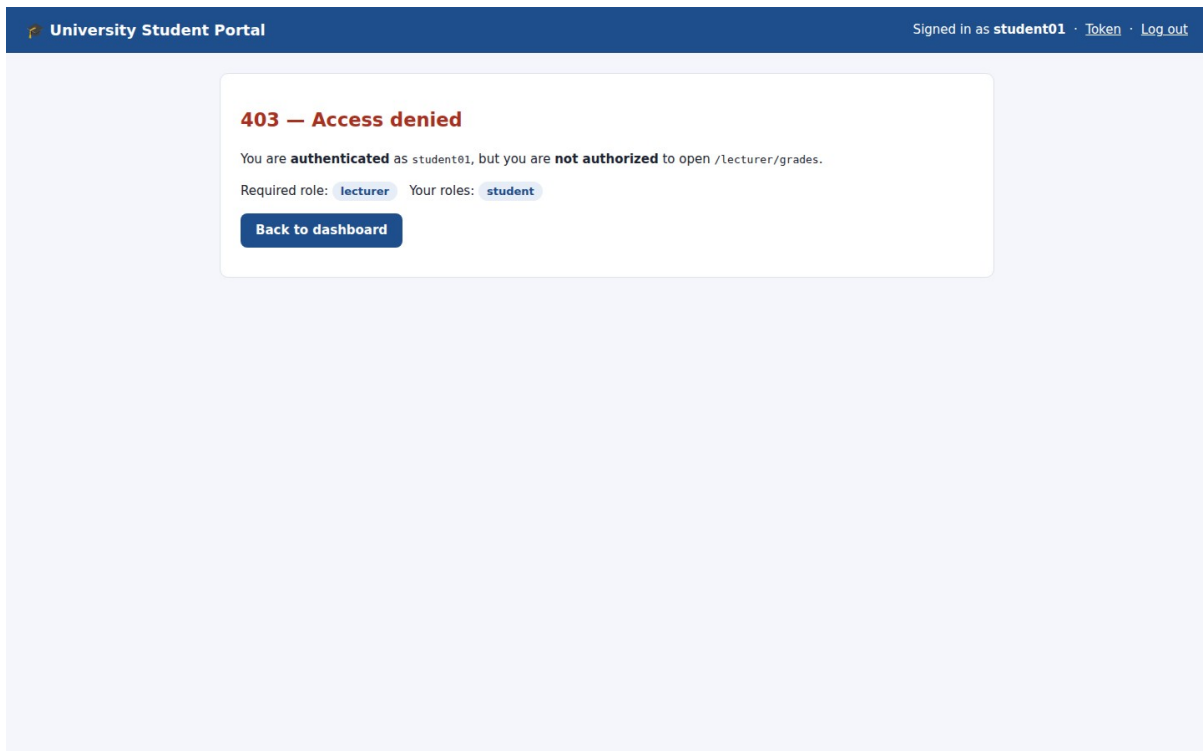
Figure 37: student01 dashboard: role student, only My Courses allowed



The screenshot shows the 'University Student Portal' header with the user 'student01' signed in. The main content area displays a green checkmark icon and the title 'My Courses & Results'. Below this, a message states: 'Access granted because your token contains the realm role: student'. A table follows with two columns: 'Course' and 'Result'. The table contains two rows: 'ISM 7101 Information Security Management' with a result of 'Pending', and 'ISM 7105 Identity & Access Management' with a result of 'A'. At the bottom of the table is a blue button labeled 'Back to dashboard'.

Course	Result
ISM 7101 Information Security Management	Pending
ISM 7105 Identity & Access Management	A

Figure 38: student01 opens /student/courses: access granted



The screenshot shows the 'University Student Portal' header with the user 'student01' signed in. The main content area displays a red error message: '403 — Access denied'. Below this, a message states: 'You are **authenticated** as student01, but you are **not authorized** to open /lecturer/grades.' It then shows 'Required role: lecturer' and 'Your roles: student'. At the bottom is a blue button labeled 'Back to dashboard'.

Figure 39: student01 opens /lecturer/grades: 403 denied

6. Client Configuration (Task 4)

A **client** in Keycloak is an application that delegates login to Keycloak. I created the client under Clients → Create client:

1. **General settings:** Client type **OpenID Connect**, Client ID `student-portal`, Name "Student Portal".
2. **Capability config:** Client authentication **On** (confidential client), Standard flow **On**; Direct access grants, Implicit flow and Service accounts **Off**; PKCE required with method **S256**.
3. **Login settings:** Root URL `http://localhost:3000`, Home URL `/`, Valid redirect URIs `http://localhost:3000/callback`, Valid post-logout redirect URIs `http://localhost:3000/`, Web origins `http://localhost:3000`.
4. **Credentials tab:** copy the generated client secret into the application's `.env` file. The secret is never committed to source control.

6.1 Authentication protocol: OpenID Connect

OIDC is an identity layer on top of OAuth 2.0. After login the application receives a signed **ID token** (a JWT) that proves who the user is, and an **access token** that carries the user's roles. I chose the **Authorization Code flow with PKCE** because it is the current best practice (OAuth 2.0 Security BCP / RFC 9700):

- the browser only ever sees a short-lived, one-time code, never the tokens;
- the code is exchanged server-to-server using the client secret **and** the PKCE code_verifier, so an intercepted code is useless;
- `state` protects against CSRF and `nonce` protects against ID-token replay;
- the application checks the token signature against Keycloak's published JWKS keys.

6.2 Redirect / allowed URLs

Redirect URIs are an **exact match with no wildcards**. Keycloak will only return an authorization code to `http://localhost:3000/callback`. This stops an attacker from building a login link that sends the code to their own site (open redirect / code theft). Web origins restricts cross-origin requests to the portal's origin, and the post-logout URI restricts where users land after signing out.

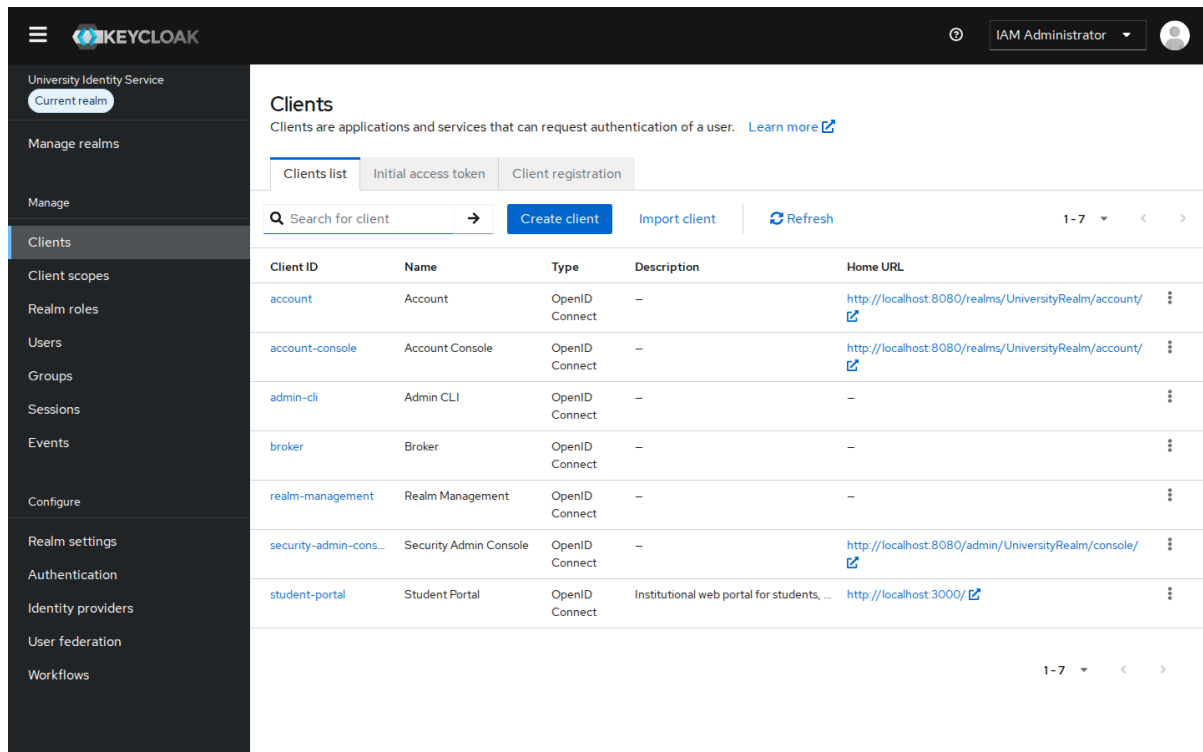


Figure 40: Clients list showing student-portal (OpenID Connect)

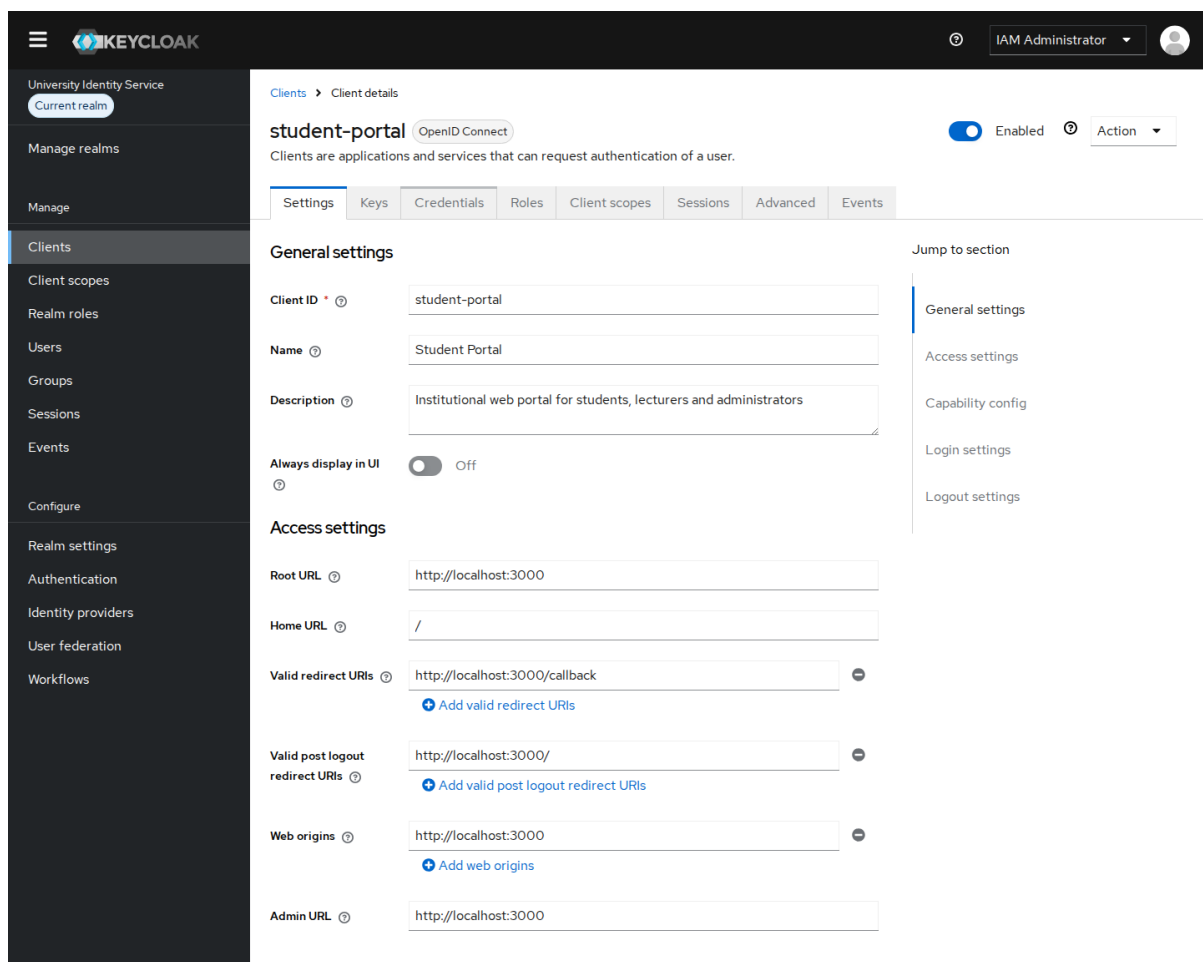


Figure 41: student-portal general and access settings: exact redirect URI, post-logout URI and web origin

Capability config

Client authentication ☒ On

Authorization ☐ Off

Authentication flow ☒ Standard flow ☐ Direct access grants

☐ Implicit flow ☐ Service account roles

☐ Standard Token Exchange

☐ JWT Authorization Grant

☐ OAuth 2.0 Device Authorization Grant

☐ OIDC CIBA Grant

Require PKCE ☒ On

PKCE Method

Require DPoP bound tokens ☐ Off

Figure 42: Capability config: client authentication On, Standard flow only, PKCE S256 required

KEYCLOAK IAM Administrator

University Identity Service
Current realm

Manage realms

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Identity providers

User federation

Workflows

Clients > Client details

student-portal OpenID Connect

Enabled Action

Settings Keys **Credentials** Roles Client scopes Sessions Advanced Events

Client Authenticator Client Id and Secret

Allowed authentication method Any

Client Secret

Save Revert

Registration access token

Regenerate

Figure 43: Credentials tab: Client Id and Secret authentication (secret masked)

GET http://localhost:8080/realms/UniversityRealm/.well-known/openid-configuration (excerpt)

```
{
  "issuer": "http://localhost:8080/realms/UniversityRealm",
  "authorization_endpoint": "http://localhost:8080/realms/UniversityRealm/protocol/openid-connect/auth",
  "token_endpoint": "http://localhost:8080/realms/UniversityRealm/protocol/openid-connect/token",
  "userinfo_endpoint": "http://localhost:8080/realms/UniversityRealm/protocol/openid-connect/userinfo",
  "end_session_endpoint": "http://localhost:8080/realms/UniversityRealm/protocol/openid-connect/logout",
  "jwks_uri": "http://localhost:8080/realms/UniversityRealm/protocol/openid-connect/certs",
  "grant_types_supported": [
    "authorization_code",
    "client_credentials",
    "implicit",
    "password",
    "refresh_token",
    "urn:ietf:params:oauth:grant-type:device_code",
    "urn:ietf:params:oauth:grant-type:jwt-bearer",
    "urn:ietf:params:oauth:grant-type:token-exchange",
    "urn:ietf:params:oauth:grant-type:uma-ticket",
    "urn:openid:params:grant-type:ciba"
  ],
  "response_types_supported": [
    "code",
    "none",
    "id_token",
    "token",
    "id_token token",
    "code id_token",
    "code token",
    "code id_token token"
  ],
  "code_challenge_methods_supported": [
    "plain",
    "S256"
  ],
  "id_token_signing_alg_values_supported": [
    "PS384",
    "RS384",
    "EdDSA",
    "ES384",
    "HS256",
    "HS512",
    "ES256",
    "RS256",
    "HS384",
    "ES512",
    "PS256",
    "PS512",
    "RS512"
  ]
}
```

Figure 44: *OIDC discovery document of UniversityRealm (issuer, endpoints, PKCE S256 support)*

7. Authentication Demonstration (Task 4)

The flow below, **User → Application → Keycloak → Login → Authentication → Application Access**, was carried out with the Student Portal:

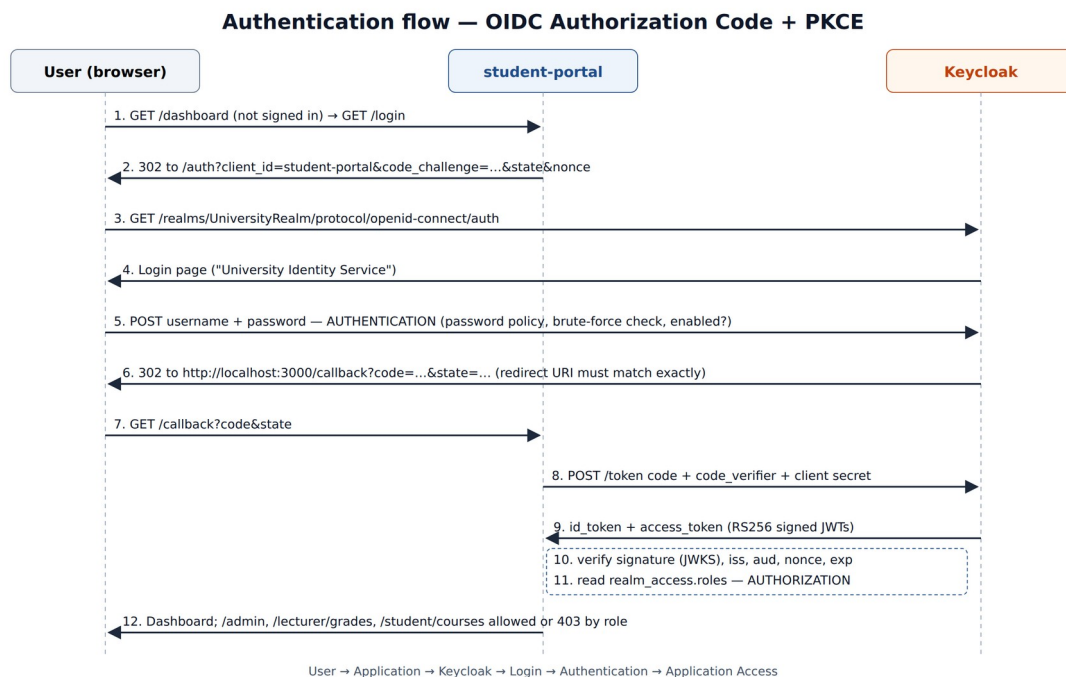


Figure 45: *OIDC Authorization Code + PKCE sequence as implemented*

1. **User → Application:** the user opens <http://localhost:3000> and clicks "Log in with University SSO".
2. **Application → Keycloak:** the portal redirects the browser to the UniversityRealm authorization endpoint with client_id, redirect_uri, PKCE code_challenge, state and nonce.
3. **Login:** Keycloak shows its own login page, branded "University Identity Service". The password is typed only into Keycloak.
4. **Authentication:** Keycloak checks the password hash, whether the account is enabled, and the brute-force counters. It then sends the browser back to </callback> with a one-time code.
5. **Token exchange:** the portal swaps the code, client secret and code_verifier for signed ID and access tokens, and validates the ID token.
6. **Application access:** the portal creates its session and shows the dashboard with the user's identity and roles.

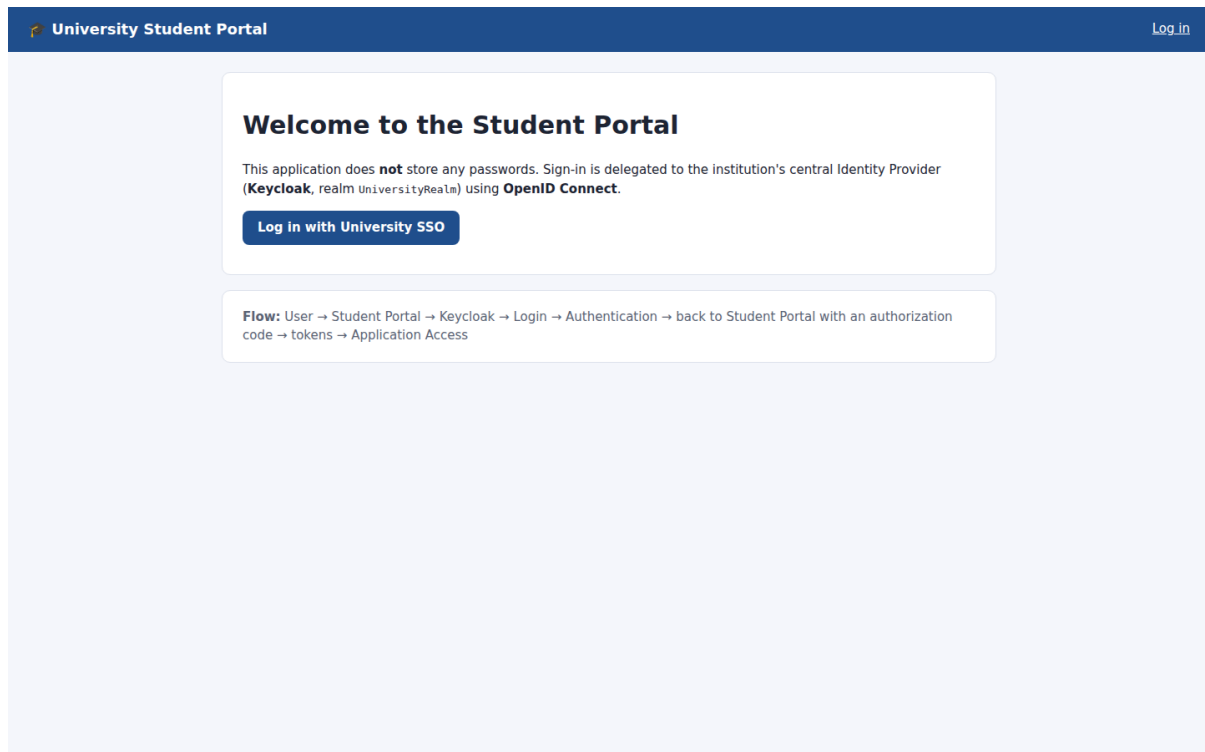


Figure 46: Step 1: Student Portal landing page (not signed in)

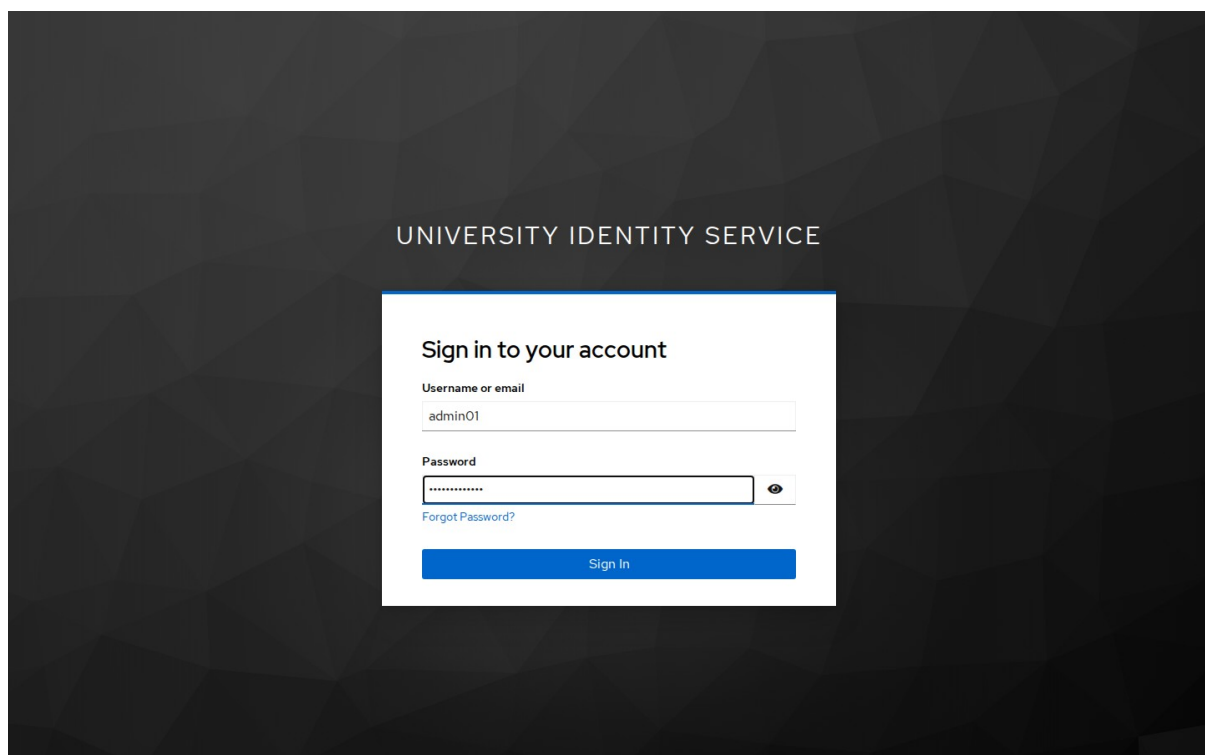


Figure 47: Step 3: redirected to the Keycloak login page for UniversityRealm

University Student Portal Signed in as **admin01** · [Token](#) · [Log out](#)

✓ Authenticated via Keycloak

Username	admin01
Name	Alice Admin
Email	admin01@university.local
Subject (sub)	6bafaa6c-0d3b-4f04-a5ad-e4a9a7e983bc
Issuer	http://localhost:8080/realms/UniversityRealm
Realm roles (authorization)	system-admin

What you are authorized to do

ALLOWED

System Administration

Manage portal configuration, audit logs and service accounts.

Requires **system-admin**

Open

DENIED

Grade Management

Enter and publish marks for the courses you teach.

Requires **lecturer**

Open

DENIED

My Courses & Results

View your registered courses, timetable and released results.

Requires **student**

Open

Figure 48: Step 6: successful authentication, identity claims issued by Keycloak shown in the application

University Student Portal Signed in as **student01** · [Token](#) · [Log out](#)

Access token issued by Keycloak (decoded payload)

The realm_access.roles claim is what the portal uses for authorization decisions.

```

{
  "iss": "http://localhost:8080/realms/UniversityRealm",
  "aud": "account",
  "azp": "student-portal",
  "typ": "Bearer",
  "sub": "1259064b-403b-4f22-b50c-bc43b930abc2",
  "preferred_username": "student01",
  "email": "student01@university.local",
  "realm_access": {
    "roles": [
      "student",
      "default-roles-universityrealm",
      "offline_access",
      "uma_authorization"
    ]
  },
  "scope": "openid profile email",
  "sid": "SrlUvElXtKl99x_RXTavh82g",
  "iat": "2026-09-28T19:53:16.000Z",
  "exp": "2026-09-28T19:58:16.000Z"
}

```

Figure 49: Decoded access token: issuer UniversityRealm, azp student-portal, realm_access.roles = [student, ...]

Sessions

Sessions are sessions of users in this realm and the clients that they access within the session. [Learn more](#)

Filter: All session types | Search session | Refresh | 1-7

User	Type	Started	Last access	IP address	Clients
admin01	REGULAR	9/28/2026, 7:53:13 PM	9/28/2026, 7:53:13 PM	127.0.0.1	student-portal
lecturer01	REGULAR	9/28/2026, 7:44:07 PM	9/28/2026, 7:44:07 PM	127.0.0.1	student-portal
lecturer01	REGULAR	9/28/2026, 7:53:14 PM	9/28/2026, 7:53:14 PM	127.0.0.1	student-portal
student01	REGULAR	9/28/2026, 7:53:16 PM	9/28/2026, 7:53:16 PM	127.0.0.1	student-portal
admin01	REGULAR	9/28/2026, 7:44:06 PM	9/28/2026, 7:44:06 PM	127.0.0.1	student-portal
lecturer01	REGULAR	9/28/2026, 7:41:26 PM	9/28/2026, 7:41:26 PM	127.0.0.1	student-portal
admin01	REGULAR	9/28/2026, 7:41:24 PM	9/28/2026, 7:41:24 PM	127.0.0.1	student-portal

1-7

Figure 50: Keycloak Sessions: active SSO sessions of the three users for student-portal

student-portal (OpenID Connect) | Enabled | Action

Clients are applications and services that can request authentication of a user.

Settings | Keys | Credentials | Roles | Client scopes | **Sessions** | Advanced | Events

Filter: Search session | Refresh | 1-7

User	Type	Started	Last access	IP address
admin01	REGULAR	9/28/2026, 7:53:13 PM	9/28/2026, 7:53:13 PM	127.0.0.1
lecturer01	REGULAR	9/28/2026, 7:44:07 PM	9/28/2026, 7:44:07 PM	127.0.0.1
lecturer01	REGULAR	9/28/2026, 7:53:14 PM	9/28/2026, 7:53:14 PM	127.0.0.1
student01	REGULAR	9/28/2026, 7:53:16 PM	9/28/2026, 7:53:16 PM	127.0.0.1
admin01	REGULAR	9/28/2026, 7:44:06 PM	9/28/2026, 7:44:06 PM	127.0.0.1
lecturer01	REGULAR	9/28/2026, 7:41:26 PM	9/28/2026, 7:41:26 PM	127.0.0.1
admin01	REGULAR	9/28/2026, 7:41:24 PM	9/28/2026, 7:41:24 PM	127.0.0.1

1-7

Figure 51: Client → Sessions for student-portal

Events
Events are records of user and admin events in this realm. To configure the tracking of these events, go to [Event configs](#). [Learn more](#)

Search events [Refresh](#) 1 - 10

Time	User ID	Event saved type	IP address	Client
September 28, 2026 at 7:53 PM	1259064b-403b-4f22-b50c-bc43b930abc2	LOGIN_ERROR	127.0.0.1	student-portal
September 28, 2026 at 7:53 PM	1f9b5d88-35bf-462e-9e34-06d4c7b63d3e	LOGIN_ERROR	127.0.0.1	student-portal
September 28, 2026 at 7:53 PM	1259064b-403b-4f22-b50c-bc43b930abc2	LOGOUT	127.0.0.1	student-portal
September 28, 2026 at 7:53 PM	1259064b-403b-4f22-b50c-bc43b930abc2	CODE_TO_TOKEN	127.0.0.1	student-portal
September 28, 2026 at 7:53 PM	1259064b-403b-4f22-b50c-bc43b930abc2	LOGIN	127.0.0.1	student-portal
September 28, 2026 at 7:53 PM	1f9b5d88-35bf-462e-9e34-06d4c7b63d3e	CODE_TO_TOKEN	127.0.0.1	student-portal
September 28, 2026 at 7:53 PM	1f9b5d88-35bf-462e-9e34-06d4c7b63d3e	LOGIN	127.0.0.1	student-portal
September 28, 2026 at 7:53 PM	6bafaa6c-0d3b-4f04-a5ad-e4a9a7e983bc	CODE_TO_TOKEN	127.0.0.1	student-portal
September 28, 2026 at 7:53 PM	6bafaa6c-0d3b-4f04-a5ad-e4a9a7e983bc	LOGIN	127.0.0.1	student-portal
September 28, 2026 at 7:44 PM	1259064b-403b-4f22-b50c-bc43b930abc2	LOGIN_ERROR	127.0.0.1	student-portal

1 - 10

Figure 52: Realm Events: LOGIN, CODE_TO_TOKEN, LOGOUT and LOGIN_ERROR (wrong password, disabled user) recorded

Negative tests

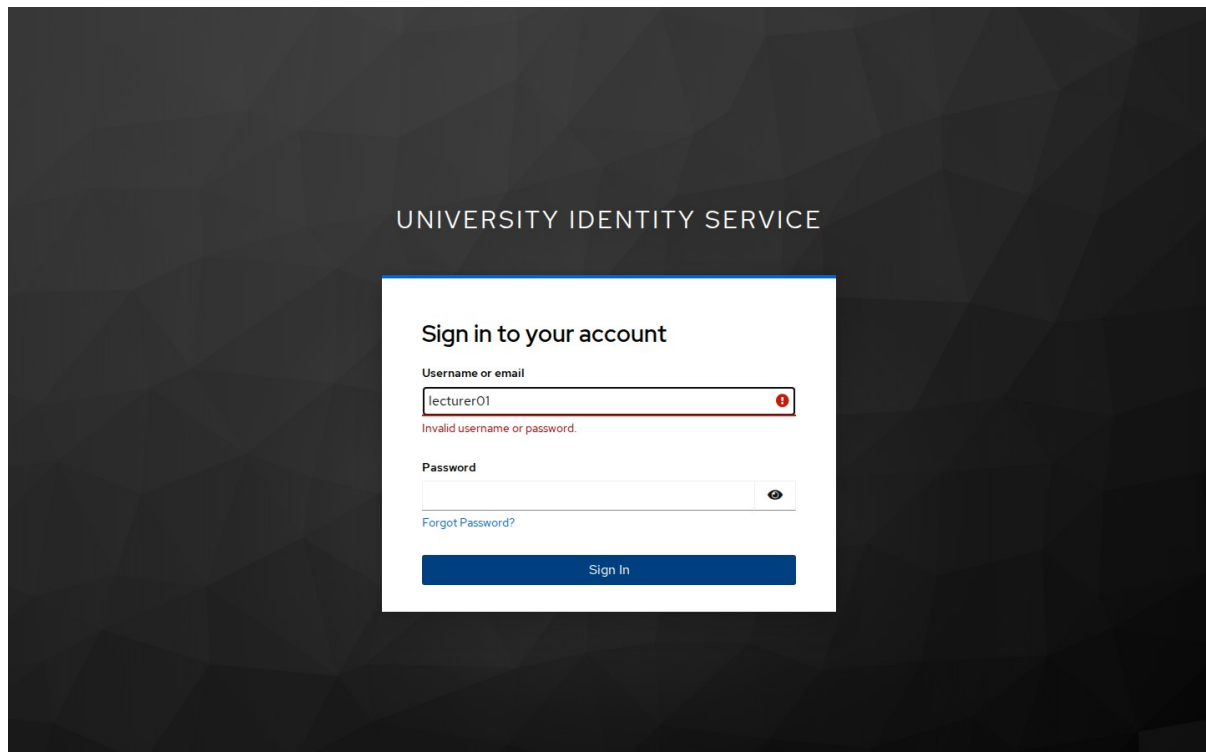


Figure 53: Wrong password: "Invalid username or password." (counted by brute-force detection)

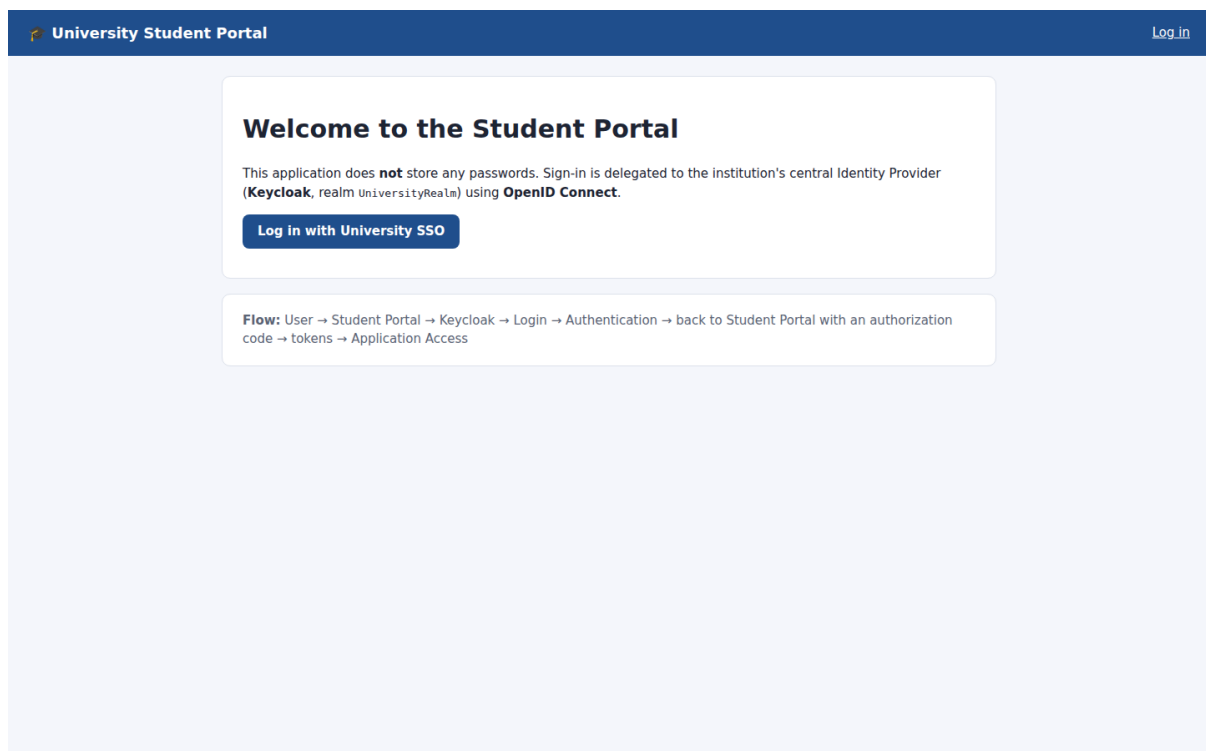


Figure 54: After logout: the portal session and the Keycloak SSO session are ended (RP-initiated logout)

7.3 Authentication vs authorization in this implementation

	Authentication	Authorization
Question	Who are you?	What are you allowed to do?

	Authentication	Authorization
answered		
Performed by	Keycloak (UniversityRealm)	Student Portal (requireRole) using roles issued by Keycloak
Inputs	Username + password, account enabled, brute-force state	realm_access.roles claim in the access token
Output	Signed ID token (sub, preferred_username, email)	Allow the page, or HTTP 403 Access denied
Failure example	Wrong password / disabled account: Keycloak refuses login (Fig. 53, Fig. 19)	student01 is logged in but gets 403 on /lecturer/grades (Fig. 39)

Put simply: **authentication proves identity once, at Keycloak.**

Authorization is checked on every request, in the application, using roles managed centrally in Keycloak. A user can be fully authenticated and still not be authorized, as the 403 pages show.

8. IAM Architecture Diagram (Task 5A)

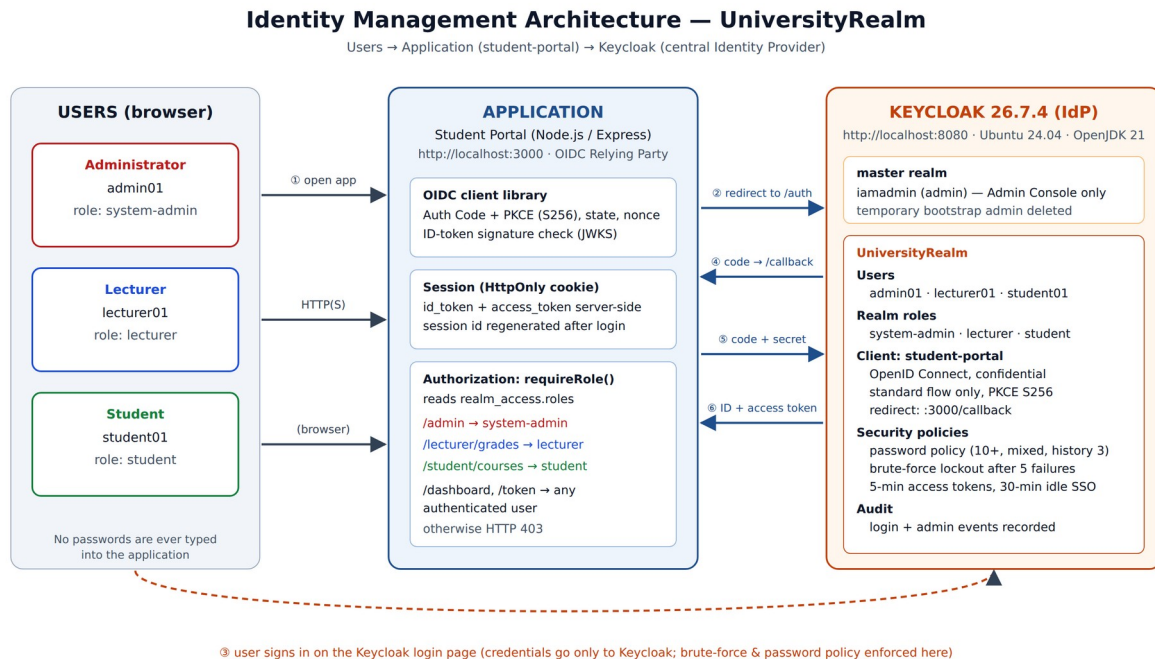


Figure 55: Identity Management Architecture: Users → Application (student-portal) → Keycloak (UniversityRealm)

- **Users** (Administrator, Lecturer, Student) use only a web browser. Their credentials go only to Keycloak.
- **Application (student-portal)** is an OIDC Relying Party. It redirects users for login, receives tokens over a back channel, and enforces role-based authorization per page.
- **Keycloak** is the central Identity Provider. The master realm is for administering Keycloak. UniversityRealm holds the users, roles, client, security policies and audit events.

9. Security Analysis (Task 5B)

Risk 1: Weak, reused or stolen passwords (credential compromise)

Risk. When every application manages its own passwords, users reuse weak passwords across systems. A leak in one system allows credential-stuffing and brute-force attacks on the others. Passwords typed into many applications can also be logged or stolen by any one of them.

How Keycloak mitigates it (as configured):

- **Single sign-on:** the password is entered only on Keycloak's login page. The portal never receives or stores it.
- **Password policy:** at least 10 characters with mixed classes, must not equal the username, and history of 3. Passwords are stored as salted hashes.
- **Brute-force detection:** after 5 failures the account is temporarily locked, with the wait growing to 15 minutes. Failures are recorded as LOGIN_ERROR events.
- Keycloak can require **MFA** (OTP or WebAuthn) and supports **passkeys**, so a stolen password alone is not enough (see Recommendation 1).

Risk 2: Excessive privileges and orphaned accounts (privilege creep)

Risk. Without central control, people pick up permissions as they change roles, and accounts of leavers stay active in some systems. An attacker or a former insider can then reach grades, personal data or admin functions they should not have. This is a common cause of data breaches and audit findings.

How Keycloak mitigates it (as configured):

- **Central RBAC:** roles are defined once in UniversityRealm and each user has exactly one role. "Users in role" gives an instant access review.
- **One switch to revoke:** disabling an account in Keycloak blocks login to **every** connected application at once (Fig. 19). Short token lifetimes make the change take effect within minutes.
- **Separation of duties:** the application administrator (admin01) cannot administer the IdP.
- **Audit trail:** login and admin events record who did what and when, for accountability and investigations.

10. Recommendations (Task 5C)

Recommendation 1: Enforce Multi-Factor Authentication (MFA), starting with privileged roles

Turn on OTP (authenticator app) or WebAuthn/passkeys in UniversityRealm. Make it **mandatory for system-admin and lecturer** using a conditional authentication flow based on role, and optional for students. Also require MFA for the master-realm `iamadmin` account. MFA stops most credential-stuffing and phishing attacks that a password policy alone cannot. Roll it out in stages: staff first, with communication and helpdesk procedures for lost devices (recovery codes).

Recommendation 2: Harden the production deployment and connect Keycloak to the authoritative directory

- Run Keycloak in **production mode** (`kc.sh start`) behind HTTPS with a valid TLS certificate. Set `sslRequired=all`, use a proper hostname, and use a PostgreSQL database with backups instead of the dev-mode H2 database. Use at least two nodes for availability.
- Configure **User Federation with LDAP / Active Directory** so accounts are created and disabled automatically from HR and admissions records (joiner/mover/leaver). This removes orphaned accounts.
- Send Keycloak login and admin events to the university **SIEM**, alert on repeated LOGIN_ERROR events and lockouts, and run **quarterly access reviews** of role membership.
- Rotate client secrets regularly, or move confidential clients to `private_key_jwt`. Keep Keycloak patched against published CVEs.

11. Conclusion

Keycloak 26.7.4 was installed and made the university's central Identity Provider. A dedicated realm, **UniversityRealm**, holds three users with full attributes. Each user has one of three realm roles, following least privilege. The **student-portal** application was registered as a confidential OpenID Connect client using the Authorization Code flow with PKCE and strict redirect URIs.

The demonstration showed the complete flow, User → Application → Keycloak → Login → Authentication → Application Access. It also showed the difference between authentication and authorization: every user signed in successfully, but each could only open the part of the portal their role allows. Disabling an account blocked access immediately, and failed logins were recorded and rate-limited.

Centralising identity in Keycloak gives the institution one place to manage, audit and revoke access. Adding MFA, production hardening and directory federation would make this lab setup ready for production.

12. References

- [1] Keycloak Project (2026). Keycloak Server Administration Guide. https://www.keycloak.org/docs/latest/server_admin/
- [2] Keycloak Project (2026). Getting started: OpenJDK / bare metal, and Configuring Keycloak for production. <https://www.keycloak.org/guides>
- [3] Keycloak Project (2026). Securing Applications and Services Guide (OIDC clients). https://www.keycloak.org/docs/latest/securing_apps/
- [4] Sakimura, N. et al. (2014). OpenID Connect Core 1.0. OpenID Foundation. https://openid.net/specs/openid-connect-core-1_0.html
- [5] Hardt, D. (2012). RFC 6749: The OAuth 2.0 Authorization Framework. IETF.
- [6] Sakimura, N., Bradley, J. and Agarwal, N. (2015). RFC 7636: Proof Key for Code Exchange (PKCE). IETF.
- [7] Lodderstedt, T. et al. (2025). RFC 9700: Best Current Practice for OAuth 2.0 Security. IETF.
- [8] Grassi, P. A. et al. (2017, rev.). NIST SP 800-63B: Digital Identity Guidelines, Authentication and Lifecycle Management. NIST.
- [9] NIST (2020). SP 800-53 Rev. 5: Security and Privacy Controls, AC-2 Account Management, AC-6 Least Privilege. NIST.
- [10] OWASP Foundation (2021). OWASP Top 10: A01 Broken Access Control; A07 Identification and Authentication Failures.

Appendix A: Reproducing the Configuration

The whole lab can be rebuilt from the project folder [ISM-Keycloak-Assignment/](#):

```
# 1. start Keycloak (see section 2.2)
KC_BOOTSTRAP_ADMIN_USERNAME=temp-admin
KC_BOOTSTRAP_ADMIN_PASSWORD=TempAdmin#2026 \
$KC_HOME/bin/kc.sh start-dev --http-port=8080

# 2. create iamadmin, UniversityRealm, roles, users, client (idempotent)
KC_HOME=/opt/keycloak-26.7.4 ./keycloak/setup-realm.sh

# 3. run the test application
cd student-portal && cp .env.example .env # paste client secret
npm install && npm start # http://localhost:3000
```

[keycloak/UniversityRealm-realm-export.json](#) contains a partial export of the realm (roles, client and policies, with secrets masked). It can be imported through Manage realms → Create realm → Resource file.

Appendix B: Marking-scheme cross-reference

Requirement	Marks	Evidence
Install & start; admin account	2	§2.2, Figs. 2–6
Version, OS, environment	1	§2.1, Fig. 1, Fig. 4
Screenshots of installation / console	2	Figs. 1–6
Create realm	2	§3.1, Figs. 7–8
Create three users	3	§4, Figs. 11–14
User attributes incl. password	1	§4 table, Figs. 12–15
Enable/disable + when	1	§4.1, Figs. 16–20
Screenshots realm & users	1	Figs. 7–20
Create three roles	3	§5.1, Figs. 21–24
Assign roles	2	§5.2, Figs. 25–30
Least privilege explained	1	§5.3
Different privileges demonstrated	2	§5.4, Figs. 31–39
Create client	3	§6, Fig. 40
OIDC protocol configured	3	§6.1, Figs. 42–44
Redirect / allowed URLs	2	§6.2, Fig. 41
Authentication demonstrated	3	§7, Figs. 45–54
Authentication vs authorization	2	§7.3
Architecture diagram	2	§8, Fig. 55
Two risks + mitigation	2	§9
Two recommendations	2	§10